

Robótica Educativa

No necesitas un curso para saber robótica.

Necesitas este libro



Si tus estudiantes pueden
aprender robótica, vos también
podés enseñarla

- ✓ Pensado para docentes SIN experiencia
- ✓ Te guía paso a paso
- ✓ Hecho por alguien que sabe cómo te sentís

👉 “No hay excusas. Ya podés empezar.”

Tabla de contenido

OBJETIVOS

Módulo 1: Fundamentos de Electricidad y Electrónica

Introducción a los conceptos básicos de electricidad y electrónica que forman la base para entender el funcionamiento de los componentes robóticos. Incluye nociones sobre corriente, voltaje, resistencia y circuitos simples.

Módulo 2: Componentes Básicos de la Robótica Educativa

Exploración de los principales componentes que conforman un robot educativo: sensores, actuadores, microcontroladores y otros elementos esenciales para el armado y la programación.

Módulo 3: Arduino y Tinkercad

Presentación del entorno Arduino y la plataforma de simulación Tinkercad para el diseño y prueba virtual de circuitos electrónicos y programación sin necesidad de hardware físico.

Módulo 4: Programación Visual por Bloques

Aprendizaje del uso de lenguajes de programación visuales basados en bloques, que facilitan la comprensión lógica y la estructura de algoritmos para principiantes.

Módulo 5: Actividades sugeridas por módulo

Propuestas prácticas y actividades diseñadas para aplicar los conocimientos adquiridos en cada módulo, fomentando el aprendizaje activo y la experimentación.

Módulo 6: Actividad STEAM: “Mi obra brilla con energía”

KITS Arduino

Este capítulo explica cómo armar kits Arduino personalizados y de bajo costo, adaptados a cada proyecto escolar. Incluye listas esenciales, comparativas y estrategias STEAM para evitar gastar en kits completos innecesarios.

Proyecto: Bocina de Patrullero

El proyecto de Patrullero con Arduino consiste en programar un buzzer y dos LEDs (rojo y azul) conectados a la placa Arduino UNO para simular la sirena y las luces de un móvil policial.



Hola!!!

Bienvenid@ a este recurso pensado especialmente para docentes que quieren iniciarse en el mundo de la electrónica aplicada a la robótica educativa, sin necesidad de materiales físicos ni conocimientos previos. Este e-book es una puerta de entrada clara y guiada al universo de la tecnología, para que vos y tus estudiantes puedan aprender creando, probando y programando desde cualquier computadora.

¿Qué vas a lograr?

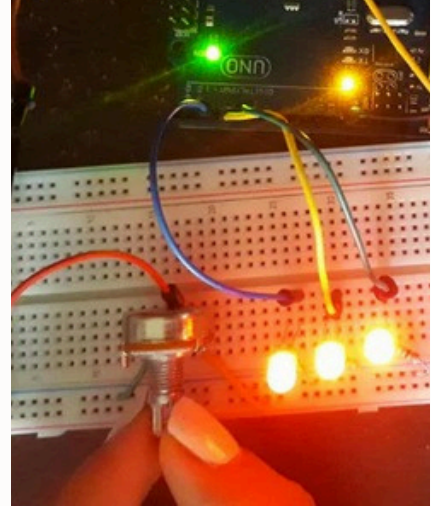
- Entender los fundamentos básicos de electricidad y electrónica.
- Conocer los componentes fundamentales de la robótica educativa.
- Usar Tinkercad para crear y simular circuitos.
- Programar con bloques de forma sencilla.

¿Para quién es este libro? Para docentes sin experiencia previa, que quieren incorporar robótica desde cero en el aula.



Objetivos

- Comprender los conceptos básicos de electrónica y electricidad.
- Identificar y conocer el uso de componentes electrónicos, especialmente diodos LED y resistencias.
- Entender la relación entre LED y resistencia, y aprender a conectarlos correctamente.
- Diferenciar y construir circuitos en serie y en paralelo.
- Armar circuitos electrónicos básicos, con y sin Arduino.
- Conocer el funcionamiento de Arduino como parte de un sistema robótico.
- Aprender a programar en bloques usando entornos como Tinkercad.
- Desarrollar lógica, pensamiento computacional y creatividad al diseñar y programar circuitos.



Módulo 1: Fundamentos de Electricidad y Electrónica

Introducción al mundo de la electrónica

La electrónica es la ciencia que controla la electricidad mediante componentes como luces, botones y sensores. Es el corazón de la robótica y de la tecnología que usamos a diario. Comprender cómo funciona nos permite no solo usar la tecnología, sino también crear y mejorar los sistemas que nos rodean. Es el primer paso para convertirnos en constructores del futuro. ¡Comencemos este viaje hacia lo invisible que da vida al mundo digital!

LA ELECTRÓNICA

La electrónica es una rama de la ciencia y la tecnología que estudia cómo usar la electricidad para controlar pequeños componentes como cables, luces, botones, sensores y chips. Gracias a ellos, podemos construir dispositivos increíbles como computadoras, celulares, autos inteligentes, videojuegos y robots. Estos componentes, cuando se conectan de forma correcta y se programan, permiten que las máquinas hagan cosas sorprendentes: encender una luz con un sensor, hacer sonar una alarma, mover un motor o enviar un mensaje automáticamente. ¡Es como hacer magia con energía! Pero en lugar de varitas, usamos circuitos y código.

ELECTRÓNICA= ELECTRICIDAD

La electricidad: el motor invisible de nuestros proyectos, es la energía que da vida a nuestros circuitos. Se mueve por los cables y activa luces, sensores, motores y otros componentes electrónicos. Cuando programamos una placa Arduino, el código indica qué hacer, pero es la electricidad la que ejecuta esas órdenes: enciende un LED, mueve un motor o hace sonar un buzzer. Sin electricidad no hay electrónica, y sin electrónica no hay robótica. Por eso, comprender cómo se genera, transmite y controla la electricidad es esencial para dominar la tecnología. Existen dos tipos de corriente eléctrica: **alterna** y **continua**, y aprender su diferencia será clave para nuestros proyectos.



Corriente Continua

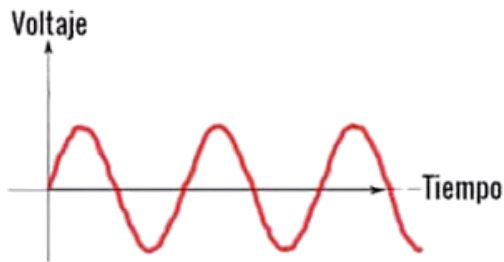
Corriente Continua (CC)

Es un tipo de electricidad que fluye en una sola dirección, como el agua que corre por una cañería.

Es la que producen las pilas y baterías, y se usa en la mayoría de los dispositivos electrónicos pequeños.

Ejemplos de dispositivos que usan corriente continua:

- Pila o batería de reloj
- Celular (funciona con CC desde su batería)
- Tablet y notebook
- Robot con Arduino
- Linterna a pilas



Corriente Alterna

Corriente Alterna (CA)

Es un tipo de electricidad que cambia de dirección muchas veces por segundo, como si fuera un columpio que va y viene.

Es la que usamos en la mayoría de los hogares y edificios, porque se puede transportar fácilmente a largas distancias.

Ejemplos de aparatos que usan corriente alterna:

- Televisor
- Lámpara conectada al enchufe
- Heladera
- Microondas
- Ventilador
- Computadora de escritorio (aunque luego transforma la corriente internamente)
- Juguetes eléctricos portátiles

Como podemos hacer que la electricidad corra y se distribuya?



Probemos hacer circular energía: Encendamos un led

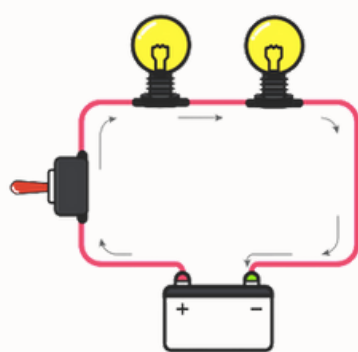
Integramos conceptos de electrónica, electricidad y componentes básicos a través de una actividad práctica.

Un circuito eléctrico se puede imaginar como una autopista por donde circula la electricidad.

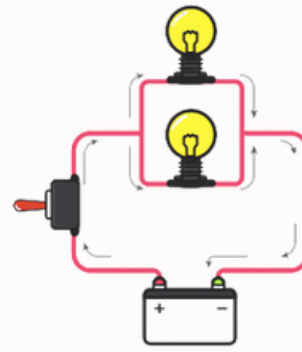
Los cables son los caminos, y los componentes (como luces LED, resistencias, motores, etc.) son como estaciones o paradas en ese recorrido.

Según cómo estén conectados los componentes, existen dos tipos de circuitos:

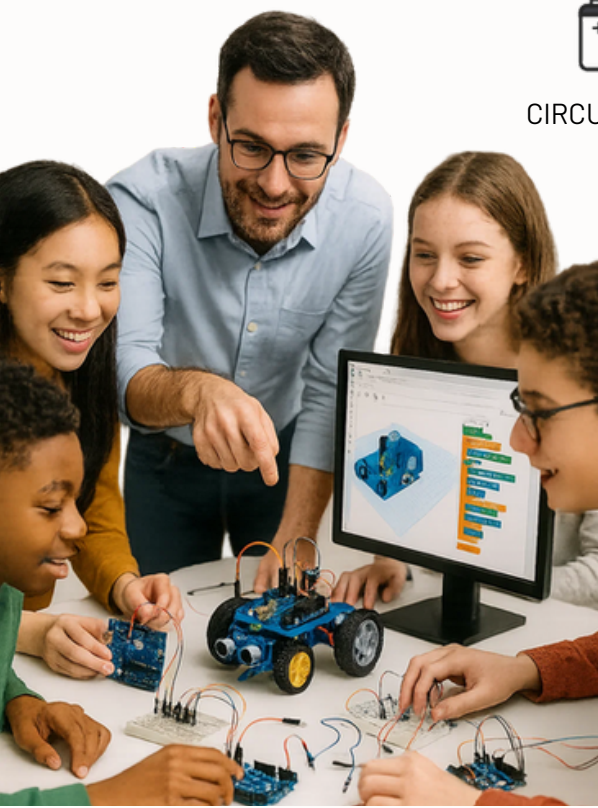
- Circuito en serie: los componentes están conectados uno detrás del otro, como los vagones de un tren. La electricidad pasa por todos ellos siguiendo un solo camino.
- Circuito en paralelo: cada componente está conectado de forma individual, como si cada uno tuviera su propia calle que lo conecta a la fuente de energía. Todos reciben electricidad de manera directa.



CIRCUITO EN SERIE



CIRCUITO EN PARALELO

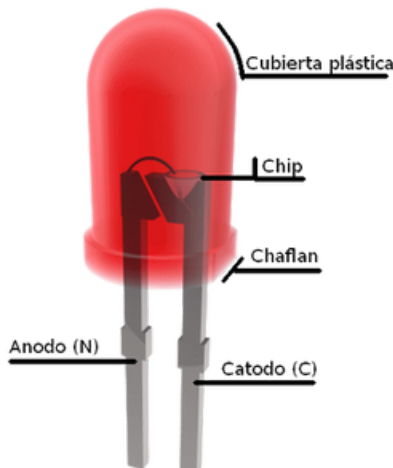


FUNCIONAN AMBOS PERO.....Brillo de los LEDs:

- En un circuito en serie, las luces LED brillan menos porque comparten la energía entre todas.
 - En un circuito en paralelo, las luces brillan más, ya que cada una recibe toda la energía que necesita.
- ¿Qué pasa si se desconecta un componente?
- En un circuito en serie, si se desconecta o se daña una parte (como un LED), todo el circuito deja de funcionar.
 - En un circuito en paralelo, si un componente se desconecta o se rompe, los demás siguen funcionando con normalidad.

¿Qué es un LED?

Partes y definicion de led



Un LED (Diodo Emisor de Luz) es un componente que emite luz cuando pasa electricidad. Gasta poca energía y dura mucho tiempo.

Se usa en controles remotos, pantallas, linternas, luces navideñas y para iluminar calles y casas.

Es muy común en los circuitos electrónicos porque permite ver cuándo pasa corriente.

Una resistencia es un pequeño componente que controla cuánta electricidad pasa por un circuito. Se usa para proteger y cuidar otros elementos, como LEDs, motores o parlantes. Por ejemplo, si conectas un LED a una batería, la resistencia evita que le llegue más electricidad de la que necesita.



El LED y la resistencia trabajan juntos como un equipo porque necesita la cantidad justa de electricidad para encenderse **sin romperse**.

La resistencia se encarga de controlar esa electricidad y proteger al LED.

Los LEDs funcionan con un voltaje bajo, entre 1,5 y 3,5 voltios, según su color y tipo.

Pero las baterías pueden tener más voltaje. Por ejemplo, una pila de 9V da mucha más energía de la que un LED necesita.

Si conectamos un LED directamente a esa pila, se puede quemar. Por eso usamos una resistencia: para bajar la cantidad de electricidad y proteger al LED.

Ejemplo:

Si tienes un LED que necesita 2V y usas una batería de 9V, la resistencia se encarga de reducir los 7V que sobran. Así, el LED recibe solo lo que necesita.

Es muy común que los estudiantes si tienen acceso a una pila y a un led los quemen, ya que no conocen de voltajes. Una pila puede alimentar desde 1,5 a 9 voltios o más. y un led (dependiendo fabricante y color) soporta desde 1,3 a 3,3 si fuera uno blanco aproximadamente.

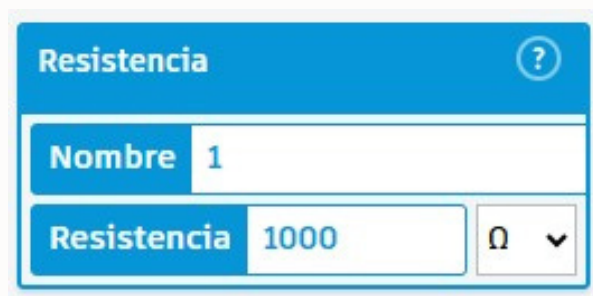
¿Qué es el valor de una resistencia? (Sirve para cuidar el LED)

El valor de una resistencia nos dice cuánta oposición pone al paso de la electricidad.

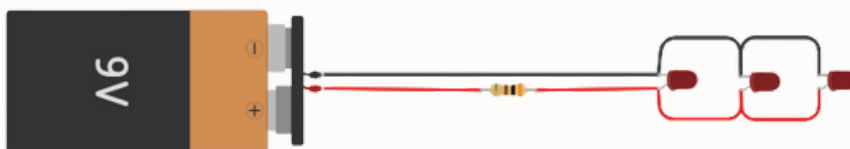
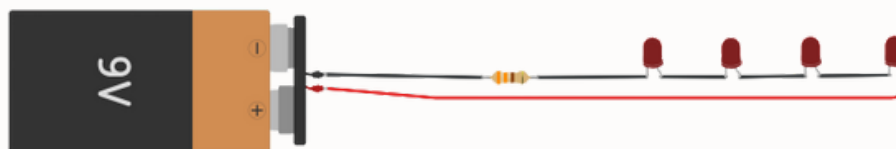
Se mide en ohmios, cuyo símbolo es Ω (se pronuncia "ohm"). Esta unidad recibe su nombre por Georg Simon Ohm, un científico que estudió cómo se mueve la electricidad en diferentes materiales.

Hay resistencias con muchos valores distintos, y se elige cada una según lo que necesite el circuito.

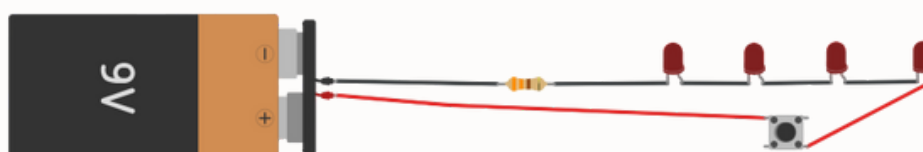
En TinkerCAD, se puede cambiar fácilmente el valor de la resistencia escribiéndolo directamente en el componente.



Te invito a que le cambies los valores en Tinkercad así pueden ver qué pasa con la intensidad de la luz de los LED al cambiar la resistencia y que crees tus circuitos en serie y paralelo como actividad



Añade un pulsador a tu circuito y mantenlo presionado en la simulación



Simulemos el encendido del LED

Conexiones de circuitos eléctricos en TINKERCAD

Vamos a conectar dos componentes esenciales en la electrónica moderna: los diodos LED y las resistencias.

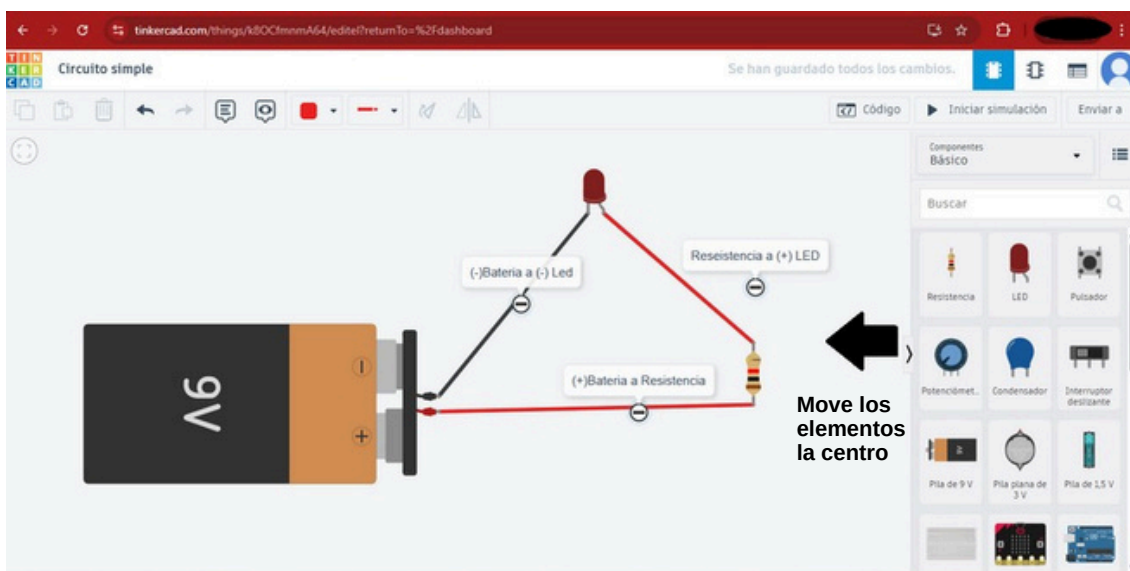
Comenzamos...

Pasos:

- Ingresa a Tinkercad.com
- Inicia sesión, Cuenta personal
- Presioná el botón crear "circuito"



Tu conexión puede quedar así



Poder mirar este tutorial



<https://www.loom.com/share/508c76b40a104c8e9f4db5e2938a5922?sid=a27fbb7d-4490-4595-9357-a24a07864f4e>

Módulo 2: Componentes Básicos de la Robótica Educativa

Nombre de los componentes mas comunes en robótica educativa

● Diodos LED

Los LEDs (diodos emisores de luz) son pequeñas luces que se encienden cuando pasa corriente por ellas. Son muy útiles para indicar si un sistema está funcionando, mostrar alertas o simplemente como decoración.

Ejemplo de aplicación: podés armar semáforos, alarmas visuales o luces que se encienden cuando presionás un botón.



● Resistencias

Son componentes que limitan la cantidad de electricidad que pasa por un circuito. Se usan para proteger otros elementos, como los LEDs, que pueden quemarse si reciben demasiada energía.

Ejemplo de aplicación: acompañan siempre a los LEDs para evitar que se fundan.



● Botones o pulsadores

Al presionar un botón, se envía una señal al Arduino que puede ser interpretada como "inicio", "paro", "cambio", etc. Es una forma muy sencilla de interactuar con un circuito.

Ejemplo de aplicación: iniciar una cuenta regresiva, cambiar el color de un semáforo, activar una luz.



● Motores DC

Estos motores convierten la electricidad en movimiento giratorio. Son ideales para mover objetos simples.

Ejemplo de aplicación: podés construir un auto seguidor de línea, un ventilador, o un robot con ruedas.



● Servomotores

A diferencia del motor común, el servomotor puede girar hasta un ángulo específico. Esto permite crear movimientos más precisos.

Ejemplo de aplicación: mover un brazo robótico, abrir una compuerta o hacer girar una torre.



● Zumbador (buzzer)

Es un pequeño componente que emite un sonido o pitido cuando recibe corriente. Se usa para alertas, alarmas o señales auditivas.

Ejemplo de aplicación: alarma que suena si se detecta movimiento o si el nivel de luz es muy bajo.



● Sensor de luz (LDR)

Este sensor detecta la cantidad de luz en el ambiente. Cuanta más luz hay, más corriente deja pasar.

Ejemplo de aplicación: encender una lámpara automáticamente cuando oscurece o medir la luz en una maqueta de ciudad inteligente.



● Sensor ultrasónico

Mide la distancia entre el sensor y un objeto usando ondas de sonido. Es muy útil para detectar obstáculos.

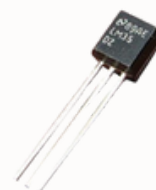
Ejemplo de aplicación: evitar que un robot choque con una pared o medir el nivel de agua en un recipiente.



● Sensor de temperatura

Detecta el calor del ambiente. Algunos sensores como el DHT11 también miden la humedad.

Ejemplo de aplicación: activar un ventilador cuando hace calor o mostrar la temperatura en una pantalla.



Módulo 3: Arduino y Tinkercad

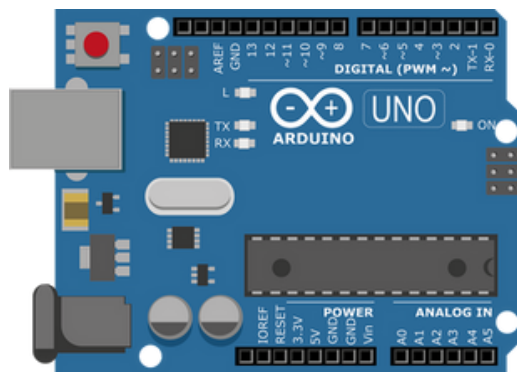
¿Qué es Arduino?

Arduino es una pequeña computadora programable, una pequeña placa física, ideal para la educación tecnológica. Puede controlar luces, sensores, motores, y responder a lo que ocurre en su entorno.

Es una plataforma muy usada en educación para aprender robótica, porque es accesible, flexible y fácil de usar.

Al programarla, podemos hacer que responda a su entorno con tareas como encender luces, seguir una línea o evitar obstáculos.

Es ideal para empezar a experimentar con la robótica porque fácil de aprender.



¿Qué es Tinkercad?

Tinkercad es un simulador en línea y gratuito que permite crear circuitos electrónicos y programarlos como si tuvieras un Arduino real.

Podés conectar cables, agregar sensores, escribir código en bloques o C++, y probar todo sin necesidad de componentes físicos.

Es una herramienta segura, intuitiva y perfecta para aprender, experimentar y detectar errores antes de construir en la vida real.



¿Qué es Tinkercad y para qué sirve?

Tinkercad es una plataforma en línea gratuita creada por Autodesk que permite diseñar, construir y simular proyectos de manera virtual. Aunque es muy conocida por el diseño 3D, una de sus funciones más potentes es la simulación de circuitos electrónicos.

Es ideal para aprender electrónica, programar placas como Arduino, y experimentar sin necesidad de materiales físicos, lo que lo hace perfecto para estudiantes y principiantes.

¿Qué se puede hacer con los circuitos en Tinkercad?

En la sección de "Circuitos" de Tinkercad, podés:

- Armar circuitos eléctricos y electrónicos con componentes como resistencias, LEDs, interruptores, motores, sensores, y más.
- Simular cómo funciona el circuito antes de construirlo en la vida real: podés ver si un LED enciende, si un motor gira, o si algo está mal conectado.
- Programar con bloques o con código (en lenguaje Arduino) para que tu circuito responda a sensores o actúe automáticamente.
- Probar errores sin dañar nada, ya que es un entorno virtual: podés equivocarte, corregir, y volver a probar todas las veces que quieras.
- Combinar componentes eléctricos con programación, lo que es ideal para proyectos de robótica, domótica, o automatización.

¿Por qué usar Tinkercad antes de armar circuitos reales?

- Porque permite experimentar sin materiales físicos, ideal si todavía no tenés componentes en clase.
- Porque ayuda a entender el comportamiento del circuito antes de conectar cables de verdad.
- Porque reduce errores, ya que se puede simular el funcionamiento y corregir antes de hacer la versión real.
- Porque fomenta el pensamiento lógico, el diseño y la solución de problemas.

Robótica en tu aula

¿QUÉ ES LA ROBÓTICA?

Es la ciencia que se encarga de crear y estudiar robots, que son máquinas que pueden hacer tareas por sí solas o con ayuda de una computadora. Estos robots se programan y luego trabajan de forma automática, sin que una persona los controle todo el tiempo.

¿QUÉ ES UN ROBOT?

Un robot es una máquina que puede hacer tareas por su cuenta, gracias a una programación previa.

Tiene partes importantes como:

- Sensores: le permiten “ver” o sentir lo que pasa a su alrededor.
- Procesador: es donde el robot “piensa” y decide qué hacer.
- Actuadores: son las piezas que se mueven, como motores o luces.

La parte más importante del robot es el microcontrolador, que actúa como su cerebro.

¿PARA QUÉ SIRVE LA ROBÓTICA?

La robótica está presente en muchos lugares, aunque a veces no la veamos.

Nos ayuda en tareas del día a día, mejora la vida de las personas y también se usa para cuidar o explorar.

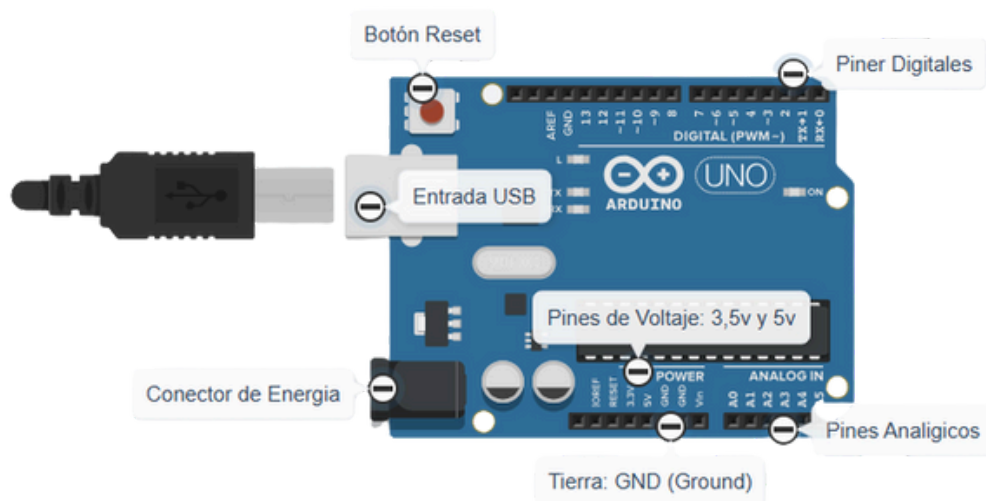
Se utiliza en:

- Industrias (como fábricas)
- Medicina (por ejemplo, en cirugías o prótesis)
- Exploración (como en el espacio o bajo el agua)
- Educación (para aprender y enseñar tecnología)

¿QUÉ ES UN ARDUINO?

Un Arduino es una pequeña computadora que funciona como el cerebro de un robot o proyecto electrónico. Por eso es nuestro aliado, también hay otras placas aplicadas a proyectos educativos, no es la única.

Se puede programar para hacer muchas cosas: encender luces, mover motores, medir temperatura o hasta crear pequeños juegos. Partes de un Arduino



Arduino + Robótica + Bloques

Lenguajes de programación

Cuando usamos Arduino, le damos instrucciones para que realice tareas como encender luces, mover motores o hacer sonar una bocina.

Para eso utilizamos un lenguaje de programación, que es como un idioma que Arduino y la computadora pueden entender.

En Tinkercad, podemos programar Arduino de dos formas:

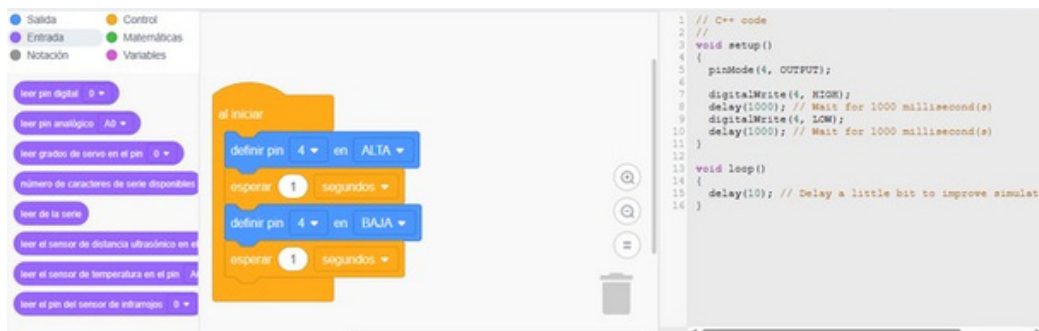
1. Escribiendo código con palabras y símbolos, en un lenguaje llamado C++.
2. Usando bloques de programación, que son piezas de colores que se conectan como si fueran un rompecabezas. Esta forma es más visual, simple y fácil de entender.

PROGRAMACIÓN EN BLOQUES

La programación por bloques en Tinkercad permite crear instrucciones sin tener que escribir código.

Solo hay que arrastrar y unir bloques de colores para construir el programa que le dice a Arduino qué hacer.

Los bloques están organizados por colores y categorías, como en Scratch 3.0. Cada color representa un tipo de acción:



DATO DE COLOR SOBRE EL PIN 13 EN ARDUINO

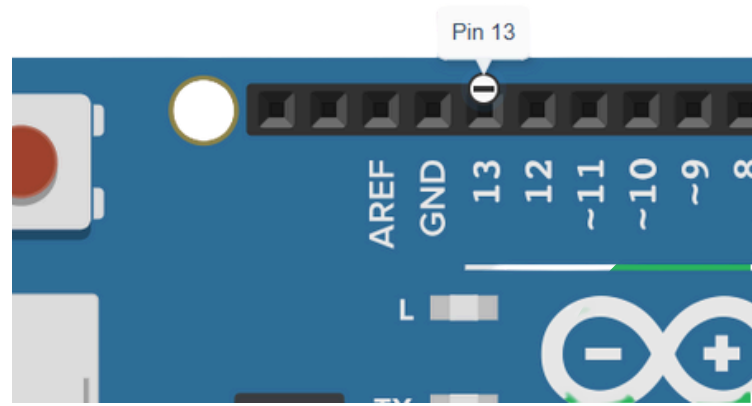
EL pin 13 es un pin especial en la placa Arduino, ya que el programa trae por defecto un código que lo usa.

Este pin está conectado a un LED incorporado en la misma placa. Gracias a eso, podemos empezar a practicar programación aunque no tengamos otros componentes.

¿QUÉ HACE EL LED DEL PIN 13?

Este LED puede encenderse y apagarse, y normalmente viene programado para parpadear:

- Se enciende por 1 segundo
- Luego se apaga por 1 segundo
- Y repite ese ciclo



Modulo 4: Programación Visual por Bloques

Cada color tiene una funcion específica...

■ Control y Condicionales (naranja)

Sirven para controlar el flujo del programa: repetir acciones, tomar decisiones o esperar un tiempo.

Ejemplos:

- “Esperar 1 segundo”
- “Repetir”
- “Si... entonces...”

◆ Salidas (celeste/turquesa)

Se usan para decirle a Arduino que active algo, como prender un LED, hacer sonar una bocina o mover un motor.

Ejemplos:

- “Fijar pin digital a ALTO”
- “Fijar velocidad del motor”

■ Entradas (violeta)

Permiten leer datos del entorno, como si Arduino “escuchara” o “viera” lo que pasa a su alrededor.

Ejemplos:

- “Leer pin digital”
- “Leer valor del sensor”

■ Matemáticas (verde)

Bloques para hacer cálculos y comparar valores.

Ejemplos:

- “+”, “-”, “>”, “<”, “==”

🌸 Variables (fucsia)

Sirven para guardar información que puede cambiar durante el programa.

Ejemplos:

- “Fijar [variable] a...”
- “Cambiar [variable] por...”

“Mostrar valor de [variable]”



¿Por qué usar bloques?

La programación por bloques en Tinkercad es una forma fácil y visual de aprender a programar. Permite entender la lógica de cómo piensa Arduino, sin preocuparse por errores de escritura. Es ideal para principiantes, y al mismo tiempo es muy poderosa para crear proyectos reales con sensores, luces, motores y más.

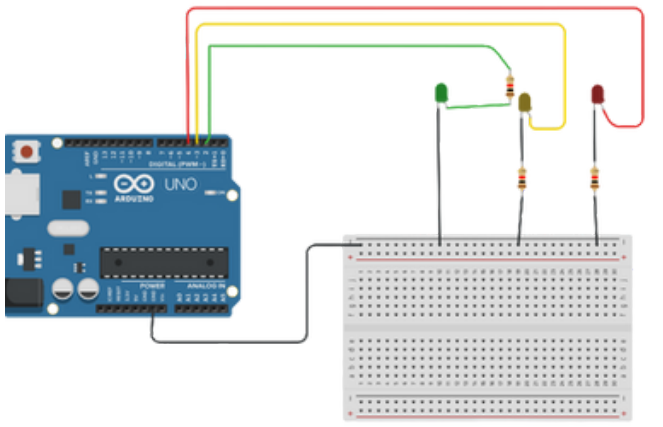
Vamos a programar nuestro led, si te animás poder hacer tu semáforo

Tutorial: Programar para encender un led y apagarlo intermitentemente

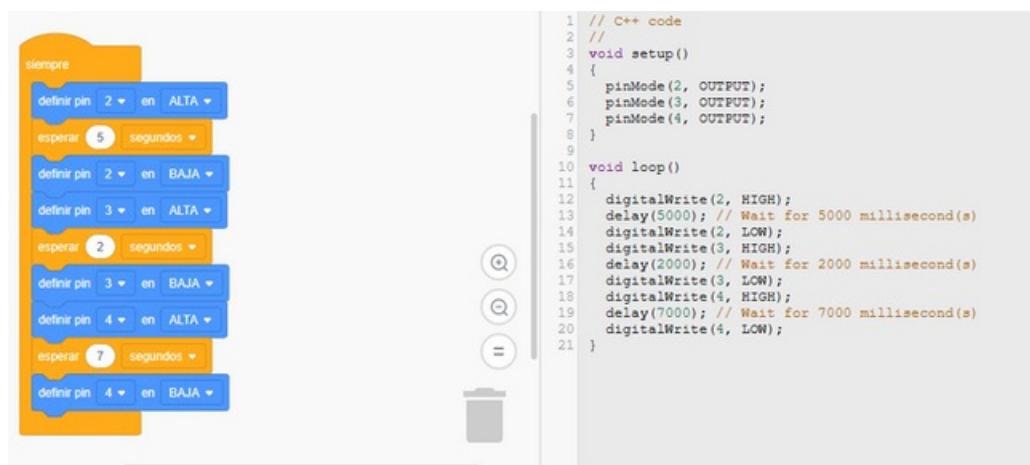
<https://www.loom.com/share/39ecfb6d90db44a093909b2a4e560075?sid=25472c86-40fc-41df-8dd7-e8c24716d36d>



Circuito electrónico



Programa en bloques y texto



Módulo 5: Actividades sugeridas por módulo

1. ¿Cuál de estos objetos tiene componentes electrónicos en su interior?

- A. Una piedra
- B. Un control remoto ✓
- C. Una hoja de papel
- D. Un libro de cuentos

2. ¿Cuál de los siguientes es un componente electrónico?

- A. Tornillo
- B. Pegamento
- C. Resistencia ✓
- D. Cartón

3. ¿Qué componente se utiliza para emitir luz en un circuito?

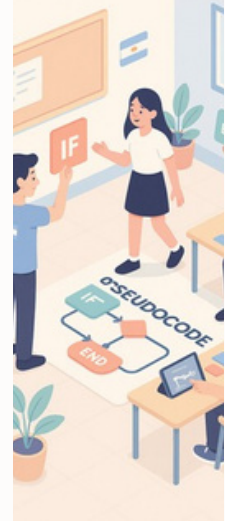
- A. Motor
- B. Zumbador
- C. LED ✓
- D. Botón

4. ¿Qué fuente de energía entrega corriente continua?

- A. Enchufe de pared
- B. Batería AA ✓
- C. Cable de internet
- D. Proyector de video

5. ¿Qué tipo de electricidad cambia de dirección muchas veces por segundo?

- A. Corriente continua
- B. Corriente alterna ✓
- C. Corriente solar
- D. Energía térmica



6. ¿Qué ocurre en un circuito si no está bien conectado?

- A. Funciona mejor
- B. No circula electricidad ✓
- C. Da más luz
- D. Se apaga el mundo

7. ¿Cuál de estos aparatos funciona con corriente alterna?

- A. Linterna
- B. Robot a pilas
- C. Ventilador de pie conectado al enchufe ✓
- D. Auto a control remoto

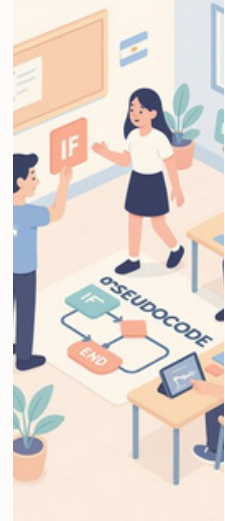
8. ¿Cuál es la función de una resistencia en un circuito?

- A. Aumentar el sonido
- B. Girar en círculos
- C. Limitar el paso de corriente ✓
- D. Generar calor

9. ¿Qué hace un botón en un circuito electrónico?

- A. Emite sonido
- B. Activa o interrumpe el paso de electricidad ✓
- C. Se ilumina solo
- D. Cambia el tipo de corriente

10. Investiga y anota al menos tres diferencias importantes entre la corriente alterna (CA) y la corriente continua (CC).



Led y Resistencias

1. ¿Qué es un LED?

- A) Un tipo de batería
- B) Un componente que emite luz con electricidad
- C) Un interruptor
- D) Un tipo de resistencia

✓ Respuesta correcta: B

2. ¿Para qué sirve una resistencia en un circuito con LED?

- A) Para encender el LED más rápido
- B) Para aumentar la luz del LED
- C) Para controlar la electricidad y proteger el LED
- D) Para apagar el LED

✓ Respuesta correcta: C

3. ¿Cuánto voltaje necesita normalmente un LED?

- A) Entre 1,5 y 3,5 voltios
- B) Más de 9 voltios
- C) Exactamente 12 voltios
- D) No necesita voltaje

✓ Respuesta correcta: A

4. ¿Qué puede pasar si conectas un LED directamente a una batería de 9V sin resistencia?

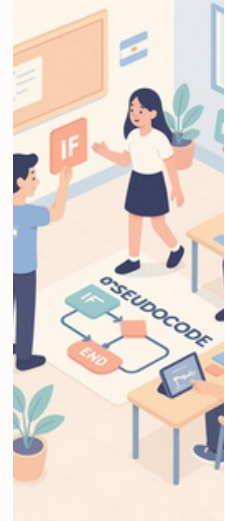
- A) El LED brillará mejor
- B) No pasará nada
- C) El LED puede quemarse
- D) La batería se recarga

✓ Respuesta correcta: C

6. Explora el concepto de resistencias eléctricas: averigua cuál es su unidad de medida y de qué manera se determina su valor.

Repaso de conceptos de Tinkercad: actividad en línea

https://es.educaplay.com/recursos-educativos/22211133-tinkercad_trivia_challenge.html



Arduino y Bloques



Verdadero o Falso

1. El pin 13 de Arduino está conectado a un LED incorporado en la placa, lo que permite hacer pruebas sin componentes externos. (V)
2. En un programa por bloques en Tinkercad, los bloques de entrada son de color celeste/turquesa. (F)
3. Los bloques de entrada son violetas.
4. La resistencia en un circuito con LED sirve para protegerlo y evitar que se queme al limitar la corriente eléctrica. (V)

¿Cuál es el color de los bloques de control y condicionales en Tinkercad?

1. a) Verde
2. b) Naranja
3. c) Fucsia
4. d) Violeta

En un programa que hace parpadear un LED, ¿qué hace el bloque o función "delay"?

1. a) Mueve el motor rápidamente
2. b) Detiene el programa por un tiempo específico
3. c) Enciende el LED
4. d) Lee un sensor

Explora el siguiente link.

Con los componentes arduino puedes usarlo para hacer repasos, distribuir materiales en trabajos prácticos, etc.

<https://wordwall.net/es/resource/34896163/ruleta-arduino>

Módulo 6: Actividad STEAM: “Mi obra brilla con energía”

Objetivos STEAM:

- Comprender el funcionamiento de un circuito eléctrico básico.
- Aplicar conocimientos de electrónica en un proyecto interdisciplinario.
- Estimular la creatividad, el trabajo en equipo y el pensamiento lógico.
- Integrar arte, tecnología y ciencia en una producción tangible.

Descripción de la actividad

Los estudiantes diseñarán una obra visual o maqueta artística (cartel, personaje, escenario, maqueta, etc.) que se ilumine mediante un circuito simple con pila, resistencia y LED, integrando conceptos eléctricos y creativos.

La obra debe tener al menos un LED funcional incorporado a través de un circuito correctamente armado y decorado con sentido estético.

Temas involucrados por disciplina

Área	Contenido trabajado
Ciencia	Flujo de corriente eléctrica, polaridad, materiales conductores, funcionamiento de un circuito eléctrico
Tecnología	Manejo de componentes (pila, resistencia, LED), diseño funcional de un circuito, simulación previa en
Ingeniería	Planificación del circuito, diseño estructural del soporte, resolución de fallas.
Arte	Estética del producto final, diseño visual del cartel/escenario, uso del color, formas y texturas.
Matemática	Cálculo básico de voltaje y resistencia (introducción), uso de escala en la maqueta, medición de distancias.

Materiales sugeridos

- Cartón, goma eva, papel madera, fibras, pegamento
- LEDs, resistencias (220Ω), porta-pilas y pilas de 9V
- Cinta adhesiva de cobre o cables para conexiones
- Tijeras, regla y marcadores



Sugerencias de organización

Pasoapaso (resumen)

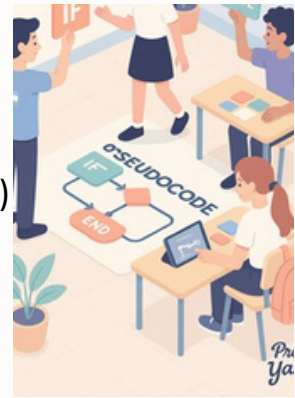
1. Investigan cómo funciona un circuito LED básico.
2. Simulan el circuito en Tinkercad para comprobar que enciende.
3. Diseñan su proyecto artístico (individual o en grupo).
4. Arman el circuito real e integran los componentes en su obra.
5. Presentan la obra explicando cómo funciona el circuito y qué representa la creación.

Evaluación (formativa)

- ✓ Comprensión del circuito (funcionamiento correcto del LED)
- ✓ Creatividad en el diseño visual
- ✓ Coherencia entre lo técnico y lo artístico
- ✓ Explicación oral del funcionamiento y propósito de la obra

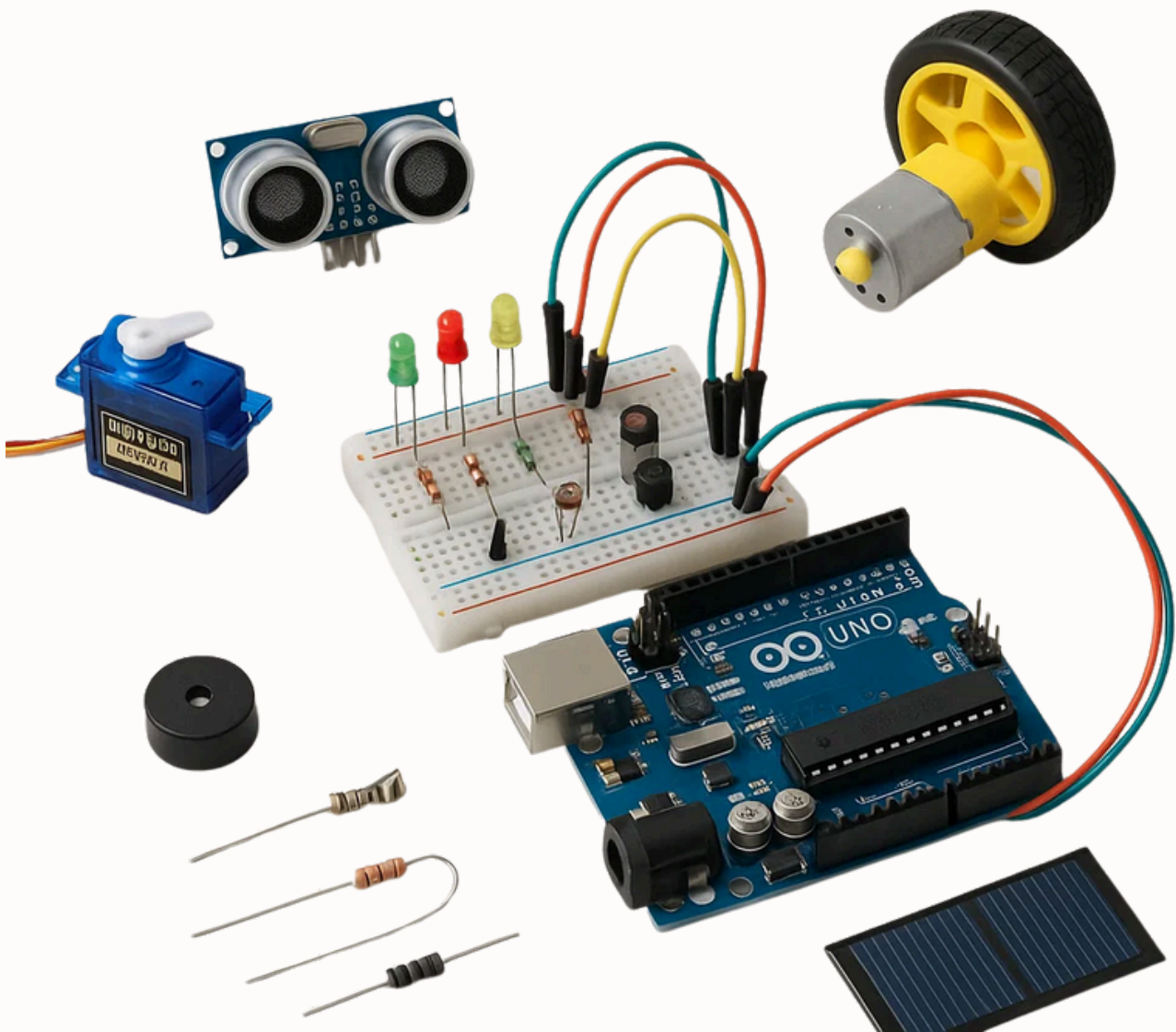
Variantes según nivel

- Inicial / 1er ciclo: usar solo LED y pila con cinta en cartulinas.
- Secundaria: agregar botón, hacer comparativa entre circuito en serie y paralelo, incorporar sensores si hay materiales.



KITS ARDUINO

**Kits inteligentes: más
aprendizaje con menos gasto**



Los famosos kits Arduino

Mariana es profesora de secundaria. El año pasado quiso enseñar robótica y buscó en internet qué necesitaba. Lo primero que apareció fue un “kit Arduino completo” de más de 40 PIEZAS. Pensó: “Sólo necesito 3 LED y un Arduino”. Pero después descubrió algo: no necesitaba todo ese kit. Con una placa, algunos LEDs y resistencias, armó un semáforo automatizado junto a sus alumnos. La inversión fue mínima y el impacto pedagógico, enorme: los chicos comprendieron electricidad, programación y hasta hicieron cuentas de tiempos en Matemática.

Moraleja: la robótica no empieza en una caja cara, empieza en un proyecto simple y accesible.



¿Qué son los kits Arduino?

Son cajas que incluyen una placa Arduino (el “cerebro” que ejecuta órdenes) más una colección de sensores, cables y pequeños motores. Se venden como “todo lo que necesitás” para empezar.

Creer que sin un kit cerrado no se puede enseñar Arduino es un error.

En la escuela, la mayoría de los componentes no se usan jamás. Lo esencial cabe en una bolsa y cuesta mucho menos.

Armar tu kit a medida

En esta sección no te vamos a dar una receta única, porque no existe un “kit universal” que sirva para todo.

Lo que sí hacemos es acompañarte a elegir el kit según el proyecto que quieras trabajar con tus estudiantes.

Un semáforo necesita poquísimos componentes.

Un robot que esquiva obstáculos requiere sensores y motores.

Una alarma escolar puede funcionar solo con un sensor de luz y un buzzer.

La clave está en definir el proyecto primero, y armar el kit después. Así ahorrarás dinero, tiempo y frustraciones.



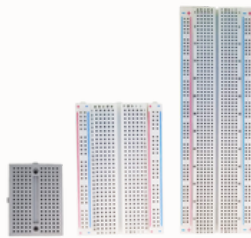
Componentes esenciales:

Estos son los básicos que no pueden faltar en casi ningún proyecto:

- Placa Arduino UNO R3 → Es el “cerebro” que ejecuta las órdenes.



- Protoboard pequeña → Tablero de pruebas que evita soldar. Hay varios tamaños, en proyectos educativos se recomienda usar los tamaños pequeños.



- Cables jumper → Los puentes que conectan cada parte.

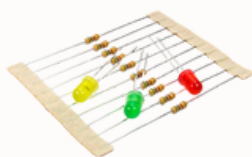
Los jumper son los cablecitos que usamos para conectar cada componente con la placa Arduino y la protoboard. Vienen en tres tipos:

- Macho – Macho 🖐️ Tienen pin en ambas puntas. Se usan sobre todo para conectar protoboard con protoboard o protoboard con Arduino.
- Hembra – Hembra 🖐️ Tienen agujerito en ambas puntas. Sirven para unir componentes que ya tienen pines salientes.
- Macho – Hembra 🖐️ Una punta con pin y la otra con agujerito. Son los más versátiles porque permiten mezclar ambos mundos (ejemplo: conectar un sensor con pines a la protoboard).



💡 **Tip pedagógico:** podés pedir a los estudiantes que comparen y clasifiquen los cables antes de empezar un proyecto, como parte de una actividad de diagnóstico. Esto refuerza vocabulario técnico de manera sencilla y práctica.

- Resistencias y LEDs → Perfectos para empezar con proyectos de luces y señales.



- Fuente de alimentación (pilas, entre 6 o 4 AA o baterías recargables).

Alimentación económica y segura

Uno de los puntos que más dudas genera es cómo darle energía a la placa Arduino.

Hay dos opciones muy accesibles:

- **4 pilas AA de 1,5 V cada una (en portapilas con cable)**
 - Total: 6V, suficiente para alimentar la placa.
 - Fácil de conseguir en cualquier kiosco o librería.
 - Inconveniente: se gastan rápido y no son recargables.
- **2 baterías recargables de litio 3,7 V (tipo 18650)**
 - Conectadas en serie, suman aprox. 7,4V.
 - Más estables y duraderas.
 - Se pueden recargar cientos de veces con un cargador económico.
 - Recomendación: ideal para uso escolar prolongado.



💡 **Tip pedagógico:** involucrar a los estudiantes en la comparación de estas dos fuentes de energía es una excelente excusa para trabajar Matemática (sumas de voltajes, proporciones) y Ciencias Naturales (tipos de energía y su impacto ambiental).

Orientación en el armado de kits ¿Qué proyecto estás pensando realizar?

No es indispensable tener un kit cerrado.

Checklist antes de invertir:

- ✓ ¿Qué proyecto quiero enseñar?
- ✓ ¿Qué componentes se repiten en varios proyectos?
- ✓ ¿Cuánto presupuesto tengo por alumno o por grupo?

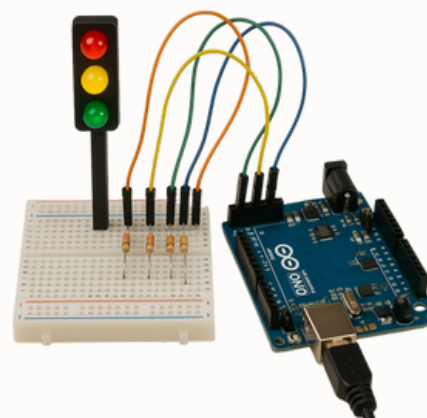
La enseñanza de robótica se basa en proyectos concretos y cada proyecto requiere componentes distintos.

Principio básico: cada proyecto define qué comprás.

1- Semáforo automatizado

Componentes:

- Arduino UNO (clon económico).
- Protoboard.
- 3 LEDs (rojo, amarillo, verde).
- 3 resistencias 220Ω.
- Cables jumper.



Descripción: Secuencia de encendido de LEDs simulando un semáforo urbano.

Enfoque STEAM: ¿Cómo podemos organizar el cruce de autos y personas en la puerta de la escuela sin que alguien tenga que estar controlando todo el tiempo?

- Ciencia: conceptos de electricidad y flujo de corriente.
- Tecnología: programación básica de secuencias.
- Ingeniería: diseño de maquetas de ciudad.
- Arte: ambientación de maqueta con cartón reciclado.
- Matemática: medición de tiempos y proporciones de intervalos.

Recomendación pedagógica

El armado del kit puede usarse como actividad didáctica en sí misma:

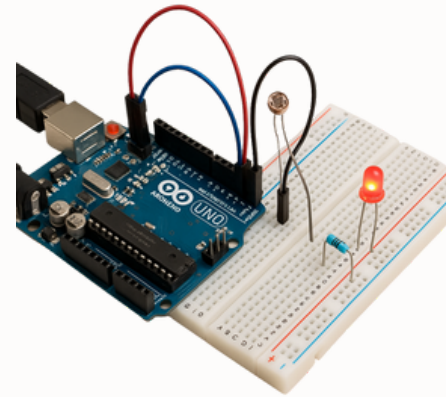
- Hacer que los estudiantes diseñen un presupuesto simulado.
- Comparar costos de comprar un kit vs. armarlo.
- Debatir sobre consumo responsable y tecnología accesible.



2- Medición de intensidad lumínica (con LDR)

Componentes:

- Arduino UNO. LDR
- (fotorresistencia). 1
- resistencia de 10kΩ.
- Protoboard. Cables
- jumper. LED indicador
- (opcional).



Descripción: El LED se enciende o apaga según la intensidad de luz ambiente.

Enfoque STEAM: ¿Cómo hago para no gastar luz si a veces dejas encendidas las lámparas de día sin darme cuenta?

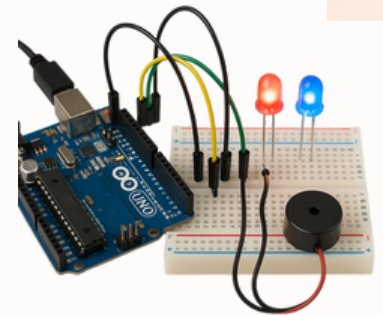
- Ciencia: relación luz-resistencia.
- Tecnología: uso de entradas analógicas en Arduino.
- Ingeniería: prototipo de lámpara automática.
- Arte: maquetas con “luces inteligentes”.
- Matemática: interpretación de valores numéricos (0–1023) del sensor.



3- Patrulla policial (LEDs + buzzer)

Componentes:

- Arduino UNO.
- 1 buzzer pasivo.
- 2 LEDs (rojo y azul).
- 2 resistencias.
- Protoboard y cables.



Descripción: Simulación de sirena policial con luces intermitentes y sonido alternado.

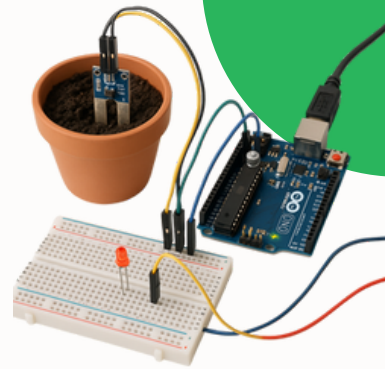
Enfoque STEAM: ¿Cómo podemos simular la sirena de un patrullero para alertar en una maqueta urbana y enseñar a los peatones a identificar señales auditivas y visuales?

- Ciencia: ondas sonoras y frecuencia.
- Tecnología: combinación de salidas digitales.
- Ingeniería: diseño de mini patrulla o maqueta de patrullero.
- Arte: efectos visuales y sonoros.
- Matemática: cálculo de tiempos de encendido y frecuencia del buzzer.

4- Sensor de humedad con LED indicador

Componentes:

- Arduino UNO.
- Sensor de humedad de suelo (analógico).
- LED indicador.
- Resistencia 220Ω.
- Protoboard, cables.



Descripción: El LED cambia de estado según el nivel de humedad detectado en la tierra.

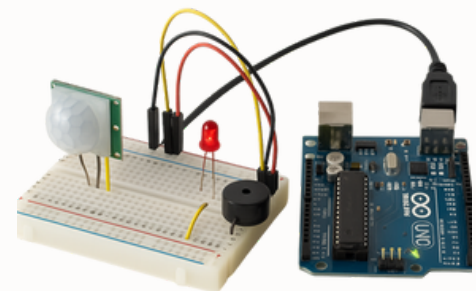
Enfoque STEAM: ¿Cómo detectar si una planta necesita agua sin tener que tocar la tierra con la mano?

- Ciencia: absorción de agua en suelos.
- Tecnología: lectura analógica.
- Ingeniería: sistemas de riego automático.
- Arte: huertas escolares creativas.
- Matemática: porcentajes de humedad en diferentes suelos.

5- Alarma con PIR (sensor infrarrojo pasivo)

Componentes:

- Arduino UNO.
- Sensor PIR.
- Buzzer.
- LED rojo.
- Resistencias y cables.



Descripción: Alarma que se activa cuando detecta movimiento humano.

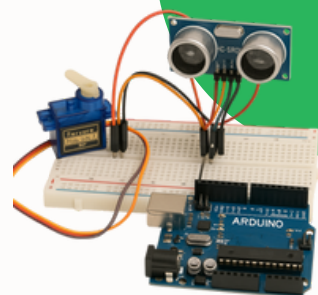
Enfoque STEAM: ¿Cómo proteger un aula o laboratorio para que suene una alarma si alguien entra fuera de horario?

- Ciencia: radiación infrarroja y calor corporal.
- Tecnología: seguridad con sensores.
- Ingeniería: prototipo de alarma escolar.
- Arte: diseño de carteles de advertencia.
- Matemática: probabilidad de detecciones correctas/falsos positivos.

6- Detector ultrasónico con servomotor

Componentes:

- Arduino UNO.
- Sensor ultrasónico HC-SR04.
- Servomotor SG90.
- Protoboard y cables.



Descripción: El servomotor gira al detectar un objeto a determinada distancia.

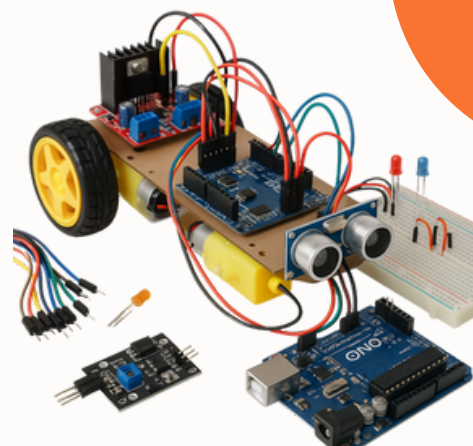
Enfoque STEAM: ¿Cómo podemos hacer que una barrera se abra sola cuando se acerca un auto, como en un estacionamiento?

- Ciencia: ondas de sonido y eco.
- Tecnología: programación con variables.
- Ingeniería: radar giratorio o barrera automática.
- Arte: maqueta futurista con radar.
- Matemática: cálculos de tiempo–distancia (velocidad del sonido).

7- Auto evita-obstáculos

Componentes:

- Arduino UNO.
- Sensor ultrasónico.
- 2 motores DC + ruedas.
- Chasis simple (cartón o acrílico).
- Driver de motores L298N.
- Batería 7,4V.



Descripción: Vehículo que detecta y esquiva obstáculos automáticamente.

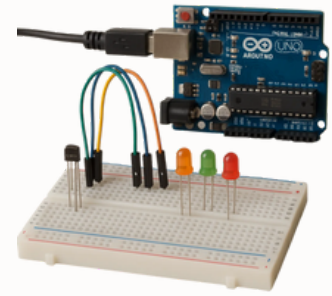
Enfoque STEAM: ¿Cómo diseñar un auto que pueda moverse por un pasillo sin chocar con mochilas, mesas o paredes?

- Ciencia: movimiento, fuerza y fricción.
- Tecnología: control de motores.
- Ingeniería: construcción de un prototipo móvil.
- Arte: personalización estética del auto.
- Matemática: cálculo de ángulos y trayectorias.

8- Detector de temperatura

Componentes:

- Arduino UNO.
- Sensor de temperatura (LM35 o DHT11).
- LED indicador o pantalla LCD.
- Resistencias, cables.



Descripción: Mide temperatura ambiente y da alerta visual o numérica.

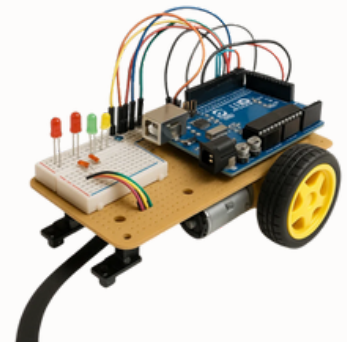
Enfoque STEAM: ¿Cómo podemos controlar la temperatura del aula para saber cuándo abrir las ventanas o prender un ventilador y no gastar de más?

- Ciencia: calor, energía térmica.
- Tecnología: lectura de sensores digitales.
- Ingeniería: estación meteorológica escolar.
- Arte: diseño de gráfica de temperaturas.
- Matemática: comparación de datos en distintas condiciones.

9- Auto seguidor de línea

Componentes:

- Arduino UNO.
- Sensores infrarrojos para detección de línea.
- 2 motores DC + ruedas.
- Driver de motores.
- Chasis básico.



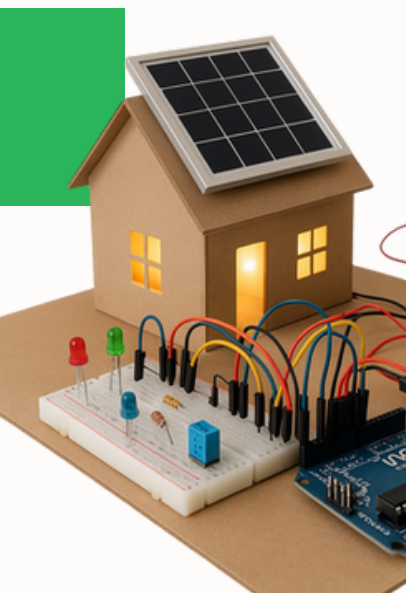
Descripción: Vehículo que sigue una línea negra sobre superficie blanca. Los infrarrojos miden la intensidad de la luz entonces guía a la motores según haya poca o mucha luz

Enfoque STEAM: ¿Cómo lograr que un auto siga un camino marcado en el piso sin que una persona lo tenga que manejar?

- Ciencia: reflexión y absorción de la luz.
- Tecnología: control de motores con sensores. Materiales conductores/aislantes
- Ingeniería: diseño de prototipos móviles.
- Arte: pistas decoradas en cartulina.
- Matemática: lógica booleana (sí/no) aplicada a sensores.



10- Casa sostenible con paneles solares



Componentes:

- Arduino UNO.
- Panel solar pequeño (5V).
- Batería recargable.
- LEDs para iluminación.
- Sensores (opcional: temperatura, luz, humedad).

Descripción: Maqueta de casa que se alimenta con energía solar y gestiona luces y sensores.

Enfoque STEAM: ¿Cómo podríamos diseñar una casa que ahorre electricidad usando energía solar para iluminarse?

- Ciencia: energía renovable, radiación solar.
- Tecnología: gestión de energía en Arduino.
- Ingeniería: diseño de sistemas sostenibles.
- Arte: maqueta arquitectónica creativa.
- Matemática: cálculo de consumo energético y eficiencia.



No hace falta llenar el aula de cajas gigantes ni gastar fortunas. Con pocos componentes bien elegidos, tus estudiantes pueden aprender robótica, programación y electrónica desde el primer día. El verdadero secreto no está en el tamaño del kit, sino en el proyecto que elijas, la creatividad con que lo guíes y el valor pedagógico que le des.

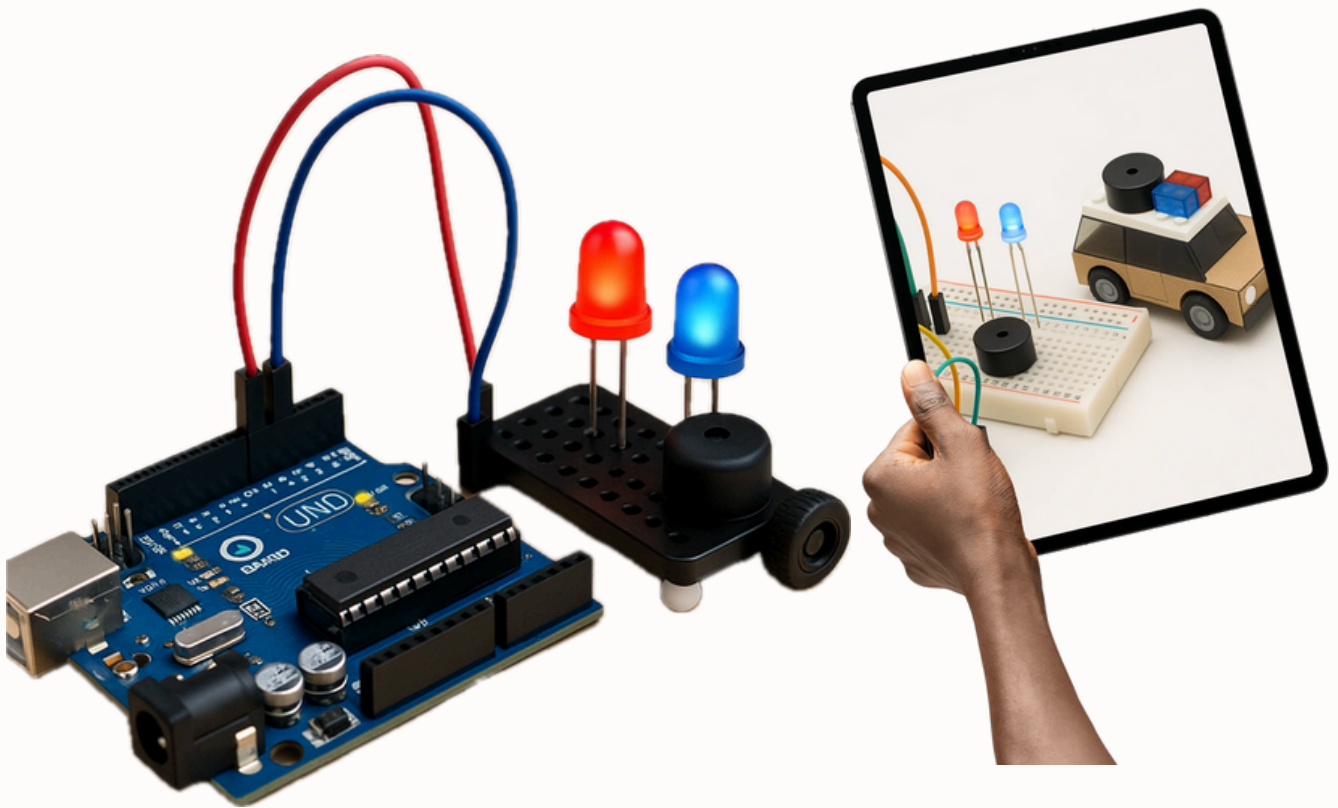
Nota al lector/a

Las imágenes que acompañan este capítulo son representaciones didácticas creadas a modo de ejemplo para ilustrar los proyectos. No corresponden a fotografías de kits reales, sino que buscan acercar visualmente la idea de cada propuesta.

Del mismo modo, las listas de materiales sugeridas tienen como objetivo orientar a los docentes en la construcción de sus propios kits educativos, optimizando recursos y evitando el gasto en kits completos que muchas veces incluyen piezas innecesarias.

El valor de este enfoque está en adaptar los materiales a cada proyecto y contexto escolar, manteniendo la accesibilidad y la creatividad como ejes centrales.

Patrullero Arduino



El proyecto Patrullero Arduino permite a los estudiantes diseñar un sistema de alerta simulado con luces tipo sirena y un buzzer, utilizando Arduino para programar su activación. A través de este proyecto, los alumnos aprenden sobre electrónica básica, programación y el montaje de circuitos, mientras crean un dispositivo interactivo de emergencia.

FUNDAMENTACIÓN

Este proyecto tiene como propósito permitir que los estudiantes reconozcan de manera concreta cómo los sistemas tecnológicos generan alertas sonoras mediante la utilización de un componente simple, como el buzzer. La propuesta se encuentra alineada con los Núcleos de Aprendizaje Prioritarios (NAP) y con los lineamientos establecidos en la Actualización Curricular de la provincia de Córdoba en relación con Cultura Digital, Programación y Robótica. A través de su implementación, se favorece la comprensión del concepto de circuito cerrado y de masa común, fundamentales en el abordaje de los sistemas eléctricos y electrónicos. Asimismo, se promueve el desarrollo del pensamiento computacional mediante la programación tanto en entornos visuales por bloques como en lenguaje de código textual. Además, el proyecto propicia la apropiación de saberes transferibles, integrando conocimientos de ciencia, tecnología, matemática, arte y creatividad, en un enfoque interdisciplinario que estimula el aprendizaje significativo y la innovación.

Objetivos

- Reconocer el buzzer y su función en un circuito electrónico.
- Comprender la necesidad de conectar todos los GND a la masa de Arduino para cerrar el circuito.
- Distinguir los pines de salida digital del Arduino (como los pines 2, 3 y 6), responsables de activar o controlar distintos dispositivos electrónicos.
- Programar el buzzer en modo sirena intermitente.
- Explicar el funcionamiento técnico del proyecto y sus posibles aplicaciones en la vida cotidiana.



Actividades

MOMENTO 1:

Exploración

- Investigamos sobre sonidos de alerta en la vida real (sirena, alarma, timbre).

Preguntar: ¿Qué tienen en común? ¿por qué son importantes?

MOMENTO 2:

Conexión del circuito

- Exploramos: qué es la “masa común” y la necesidad de unir los GND.

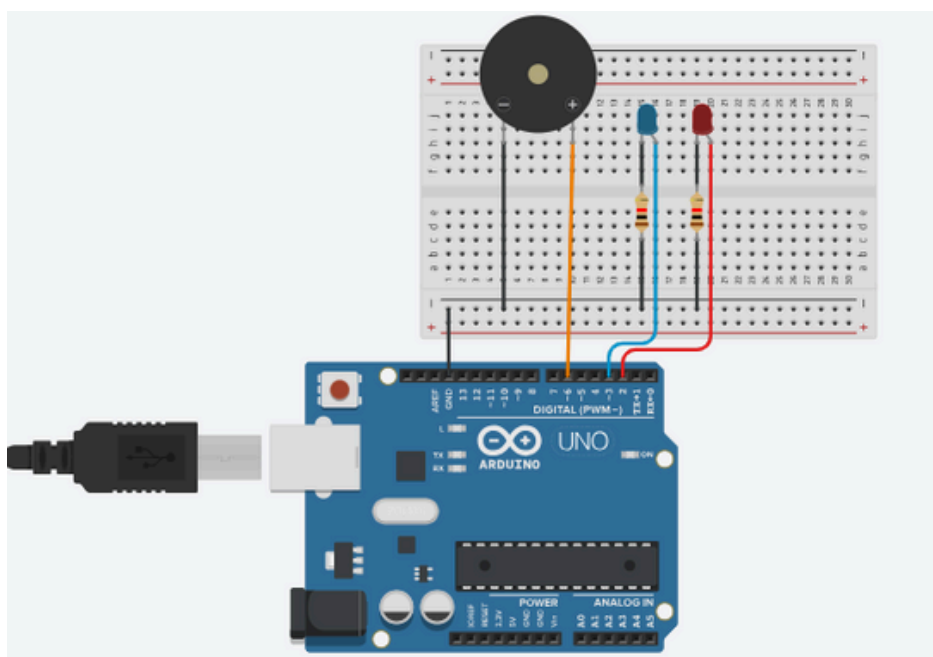
Hacemos nuestro circuito: Arduino + buzzer “Activo”

(Tutorial)

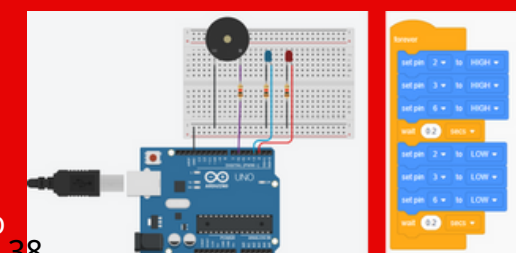


Materiales

- Arduino UNO
- Protoboard.
- LED rojo
- LED azul
- Buzzer (activo o pasivo).
- Cables jumper tipo dupont (en caso de hacerlo físico)
- Resistencia (si se utiliza buzzer pasivo y LED).
- Fuente de alimentación o cable USB. (en caso de hacerlo físico)



En este proyecto usaremos buzzer activo ya que nos permite cambiar el tono. En buzzer pasivo requiere resistencia en caso de usarlo. En esta imagen te muestro uno pasivo



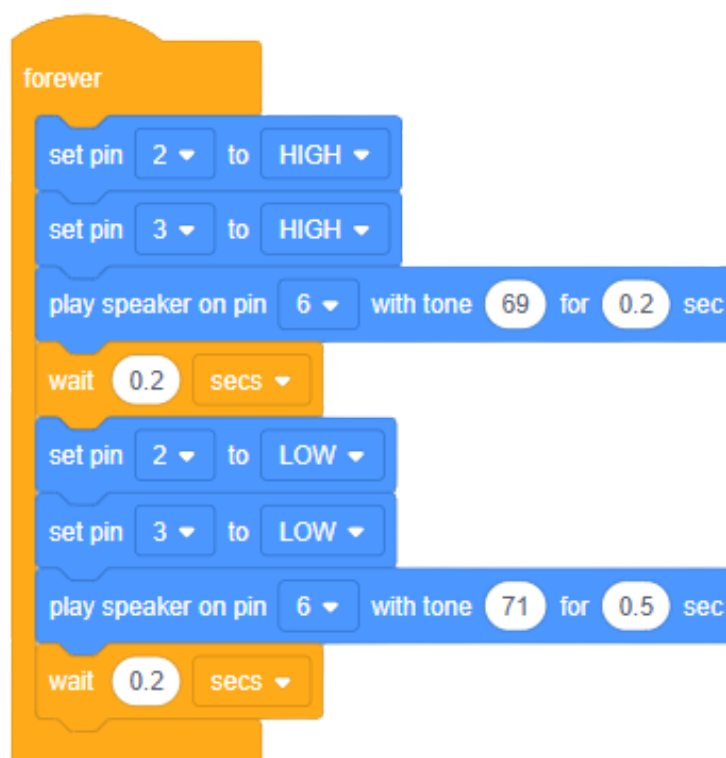
A PENSAR!!

1. ¿Qué función cumple cada uno de los pines donde conectaste los LED (pines 2 y 3)?
2. ¿Por qué es importante respetar la polaridad al conectar un LED al Arduino?
3. ¿En qué pin se conectó el buzzer y por qué se eligió ese?
4. ¿Qué diferencia hay entre un buzzer activo y uno pasivo, y cómo influye eso en la conexión?
5. ¿Por qué es recomendable usar una resistencia al conectar un buzzer pasivo?
6. ¿Qué rol cumple la protoboard en el armado del circuito?
7. ¿Cómo se aseguran de que todos los componentes compartan una masa común (GND)?
8. ¿Qué tipo de pines (digitales, analógicos, PWM) se utilizaron y por qué?
9. ¿Qué ocurriría si se invierte la conexión de un LED o del buzzer?
10. ¿Cómo verifican si las conexiones están correctas antes de subir el programa?

PROGRAMACION

Qué hará nuestro programa?

Encender dos LED conectados a los pines digitales 2 y 3 del Arduino, y al mismo tiempo generar sonidos con un buzzer conectado al pin 6.



Instrucciones paso a paso para crear este código en bloques

1. Abrió el entorno de programación por bloques (Tinkercad Circuits).
2. Buscó el bloque de bucle "por siempre" (forever). Este bloque se encuentra generalmente en la categoría "Control". Colocó todo el código dentro de este bucle, ya que queremos que se repita continuamente.
3. Encendido de LED
 - a. Usó dos bloques del tipo set pin (n) to (HIGH).
 - b. Configuró el pin 2 y el pin 3 en estado HIGH, lo que hará que los LED se enciendan.
4. Reproducción de sonido
 - a. Agregó un bloque que diga algo como play speaker on pin (n) with tone (note) for (duration).
 - b. Usó el pin 6 y colocó el tono 69 (corresponde a la nota A5, 880 Hz) durante 0.2 segundos.
5. Pausa
 - a. Agregó un bloque wait 0.2 seconds para mantener el estado unos instantes.
6. Apagado de LED
 - a. Usó nuevamente dos bloques set pin (n) to (LOW), para apagar los LED.
7. Segundo sonido
 - a. Agregó otro bloque play speaker on pin 6 with tone 71 for 0.5 sec
 - b. El tono 71 corresponde a la nota B5 (988 Hz).
8. Otra pausa
9. Nuevamente, agregó wait 0.2 seconds para cerrar el ciclo.

Extensión creativa

- Variar la frecuencia del tono para crear una melodía.
- Incorporar un LED que parpadee al mismo ritmo.
- Diseñar una carcasa con cartón o impresión 3D que imite una sirena. (Realizar diseño 3D en TINKERCAD)

Cierre y reflexión

- Puesta en común de variantes creadas.
- Reflexión: ¿dónde encontramos sistemas de alerta en la vida real? ¿Qué pasaría si no existieran?

Autoevaluación: Circuito + Programación

Sobre el armado del circuito

1. ¿Conecté correctamente los LED respetando su polaridad (ánodo y cátodo)?
2. ¿Verifiqué que todos los componentes comparten una misma conexión a GND (masa)?
3. ¿Usé el pin correcto para conectar el buzzer según lo que indicaba el programa?
4. ¿Funcionó el circuito al cargar el programa o tuve que revisar alguna conexión?

Sobre la comprensión del programa

1. ¿Entiendo qué hace cada bloque o línea del programa?
2. ¿Puedo explicar por qué se usó tone() o el bloque de sonido con frecuencia y duración?
3. ¿Sé cómo cambiar el tono o el tiempo del buzzer y qué efecto tendría?
4. ¿Reconozco cómo los wait afectan la secuencia del programa?

Sobre el proceso de trabajo

1. ¿Revisé las conexiones y el código antes de subirlo al Arduino?
2. ¿Pude resolver los errores o dificultades que aparecieron durante el armado o la programación?

Sugerencia para estudiantes:

"Marcá con ✓ las preguntas que puedas responder con seguridad. Las que no, ¡son una buena oportunidad para repasar!"

Rúbrica de Evaluación

Criterio	Excelente (4)	Bueno (3)	Satisfactorio (2)	En proceso (1)
Armado del circuito	El circuito está completo, bien conectado, y funciona sin errores.	El circuito está correctamente armado con mínimas fallas corregidas.	El circuito tiene errores, pero logra cumplir parcialmente su función.	El circuito presenta conexiones incorrectas y no funciona.
Comprensión de los componentes	Identifica y explica claramente el rol de cada componente (LED, buzzer, El código está bien estructurado, completo y ejecuta correctamente. Demuestra una	Reconoce la mayoría de los componentes y su función.	Reconoce algunos componentes, pero con confusión.	Tiene dificultades para identificar o explicar los componentes.
Programación en bloques	El código funciona con detalles menores a corregir.	El código presenta errores que dificultan la ejecución correcta.		El código está incompleto o no funciona.
Relación código-circuito	comprensión clara de cómo el programa afecta el circuito. Trabaja de forma autónoma, resuelve problemas	Relaciona correctamente e la mayoría de las instrucciones con el funcionamiento	Tiene dificultad para vincular el código con el resultado en el circuito.	No logra establecer relación entre el programa y el circuito.
Autonomía y resolución de problemas	Trabaja con cierta autonomía y busca soluciones	Requiere apoyo constante para avanzar.		Tiene dificultad para avanzar sin asistencia directa.

Respuestas orientadoras a las preguntas 1

1. **¿Qué función cumple cada uno de los pines donde conectaste los LED (pines 2 y 3)?** Los pines 2 y 3 funcionan como salidas digitales que encienden o apagan los LED mediante señales eléctricas.
2. **¿Por qué es importante respetar la polaridad al conectar un LED al Arduino?** Porque el LED solo permite el paso de corriente en un sentido. Si se conecta al revés, no enciende.
3. **¿En qué pin se conectó el buzzer y por qué se eligió ese?** En el pin 6, porque permite usar la función `tone()` para generar sonidos a distintas frecuencias.
4. **¿Qué diferencia hay entre un buzzer activo y uno pasivo, y cómo influye eso en la conexión?** El activo ya tiene un oscilador interno y suena al aplicar corriente. El pasivo necesita una señal de frecuencia, por eso requiere programación (como `tone()`).
5. **¿Por qué es recomendable usar una resistencia al conectar un buzzer pasivo?** Para limitar la corriente, proteger el buzzer y el pin del Arduino, y evitar sobrecalentamiento.
6. **¿Qué rol cumple la protoboard en el armado del circuito?** Facilita soldar, facilitando pruebas y cambios.
7. **¿Cómo se aseguran de que todos los componentes compartan una masa común (GND)?** Conectando todos los negativos o tierras (GND) de los componentes al GND del Arduino.
8. **¿Qué tipo de pines (digitales, analógicos, PWM) se utilizaron y por qué?** Se usaron pines digitales (2, 3 y 6) porque los LED y el buzzer solo necesitan encendido/apagado o señales digitales.
9. **¿Qué ocurriría si se invierte la conexión de un LED o del buzzer?** El componente no funcionará. En el peor caso, podría dañarse si no se protege correctamente.
10. **¿Cómo verifican si las conexiones están correctas antes de subir el programa?** Revisando que cada componente esté en el pin correcto, que la polaridad sea la adecuada, y que todas las conexiones estén firmes y bien ubicadas.

Respuestas orientadoras a las preguntas 2

- **¿Qué sucede primero en la secuencia: se encienden los LED o suena el buzzer?** Primero se encienden los LED, y luego suena el buzzer con el primer tono. **¿Qué función cumple el bloque wait en este programa?** Hace que el programa espere un momento antes de pasar al siguiente paso, permitiendo ver y oír claramente cada acción. **¿Por qué se usan los bloques set pin (n) to HIGH y LOW?**
- **¿Qué efecto tienen en los LED?** El bloque HIGH enciende el LED y LOW lo apaga, controlando el paso de corriente. **¿Qué diferencia notaste entre los dos tonos que emite el buzzer?** El segundo tono es más agudo y largo que el primero, porque tiene una frecuencia y duración mayores. **¿Qué pasaría si quitamos el segundo bloque wait al final del programa?** El cambio entre sonidos y luces sería más rápido, y la secuencia se vería menos clara o más confusa.

Anexo: Propuesta STEAM

Pregunta impulsora

¿Cómo podríamos construir una alarma sonora y luminosa que avise en la escuela sin gastar dinero en dispositivos comerciales? *(Invita a pensar en ciencia del sonido, tecnología de buzzers y luces, y soluciones creativas con materiales simples).* **¿De qué manera podemos transformar un simple buzzer y dos LEDs en una “sirena” que combine música, matemática y arte?** *(Integra ritmo, tiempos en ms, diseño de secuencias y performance escolar).*

ÁREAS STEAM

Ciencia

- Explorar cómo el sonido se propaga a través de diferentes materiales.
- Relacionar frecuencia con altura del sonido.

Tecnología

- Comparar buzzer activo vs. pasivo en cuanto a código y funcionamiento.
- Relacionar con alarmas reales.

Ingeniería

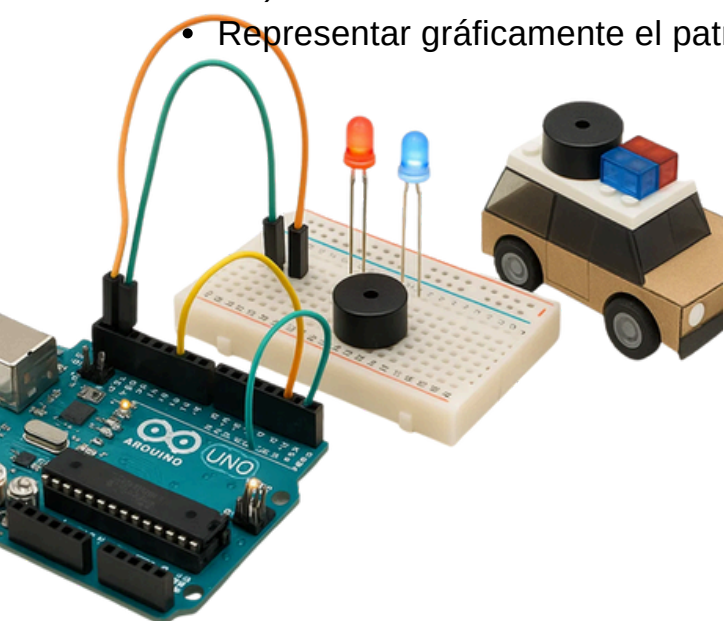
- Diseñar un soporte para el buzzer con cartón o impresora 3D.
- Montar el circuito en un prototipo de patrulla de cartón/Lego.

Arte

- Crear una secuencia musical corta que simule una sirena.
- Integrar luces y sonido en una “performance escolar” de alerta.

Matemática

- Calcular tiempos y frecuencias (ejemplo: ciclo de encendido/apagado en ms).
- Representar gráficamente el patrón rítmico del buzzer.



PROGRAMACIÓN Y ROBÓTICA

ARDUINO

SIMULACIONES EN TINKERCAD

BONUS



TECNOMAKETERS

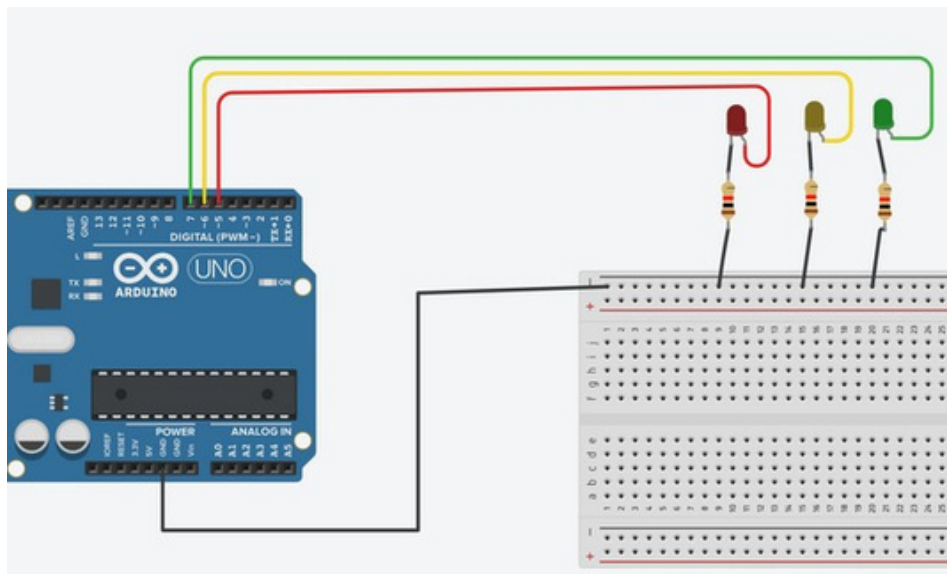
CIRCUITOS BONUS

SEMÁFORO

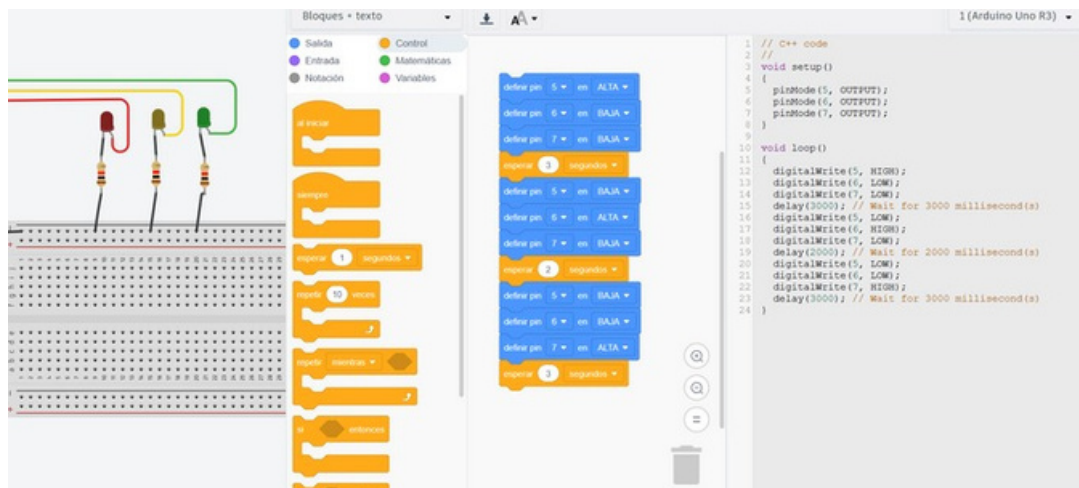


- ✓ Arduino UNO
- ✓ LED rojo
- ✓ LED amarillo
- ✓ LED verde
- ✓ Resistencias (3 de 220 Ω , una para cada LED)
- ✓ Jumpers (cables de conexión)
- Protoboard (opcional, para mayor orden)

Iniciá la simulación al terminar



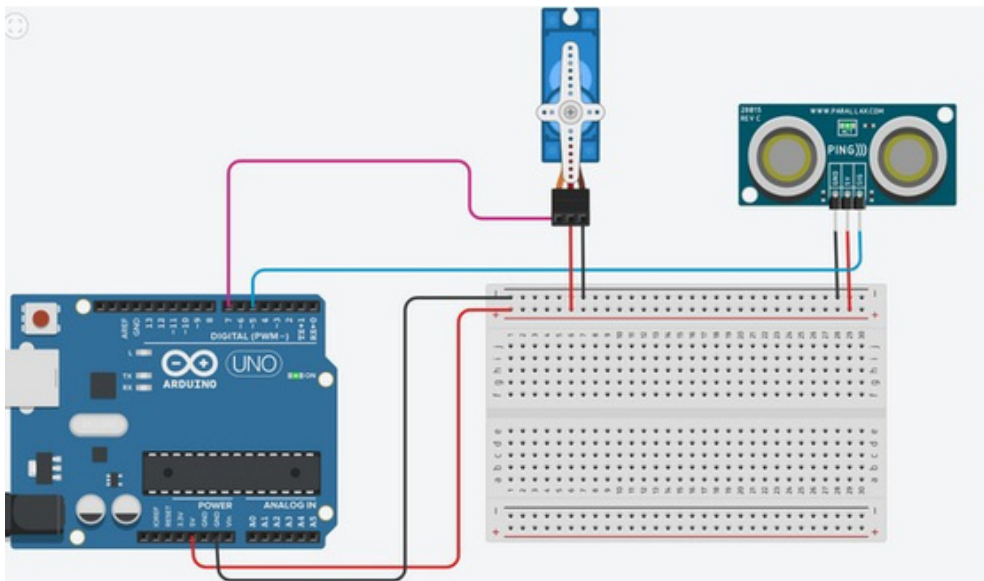
CÓDIGO EN BLOQUES Y TEXTO



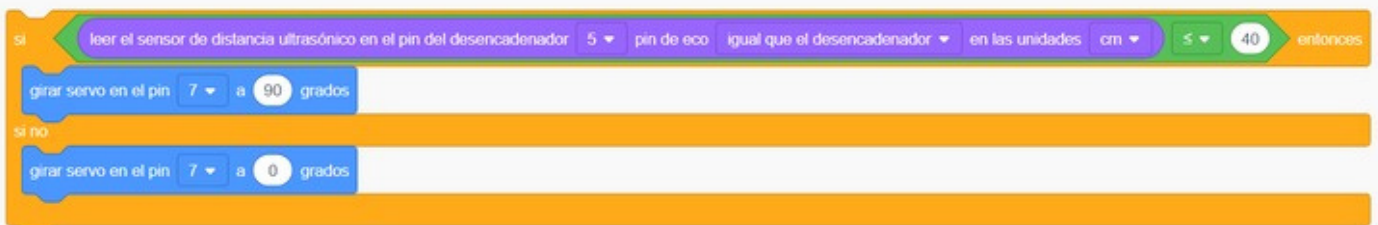
BARRERA DE ESTACIONAMIENTO

- ✓ Arduino UNO
- ✓ Servo motor
- ✓ Sensor ultrasonido (HC-SR04) para detectar la proximidad del coche
- ✓ LED (opcional, para indicar que la barrera está levantada o bajada)
- ✓ Resistencia (220 Ω para el LED, si lo utilizas)
- ✓ Jumpers (cables de conexión)
- ✓ Protoboard (opcional, para mayor orden)

*Iniciá la simulación
terminar*



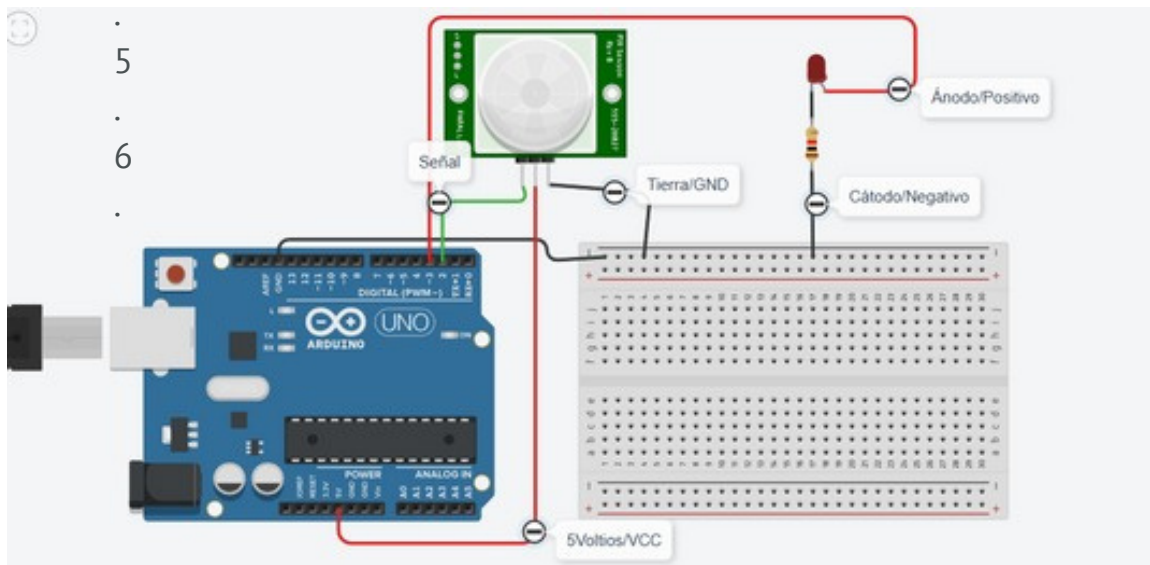
CÓDIGO EN BLOQUES Y TEXTO



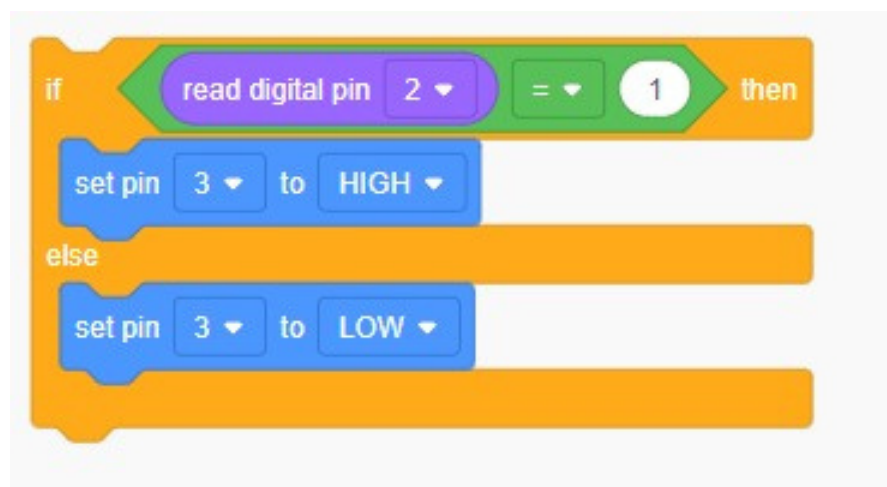
DETECTOR DE MOVIMIENTO

Materiales necesarios(en Tinkercad):

- 1 ☒ Arduino UNO
- . ☒ Sensor PIR (sensor de movimiento)
- 2 ☒ LED (opcional, para ver si el PIR detecta)
- . ☒ Resistencia (220 Ω para el LED)
- 3 ☒ Jumpers (cables de conexión)
- . ☒ Protoboard (opcional, para mayor orden)
- 4

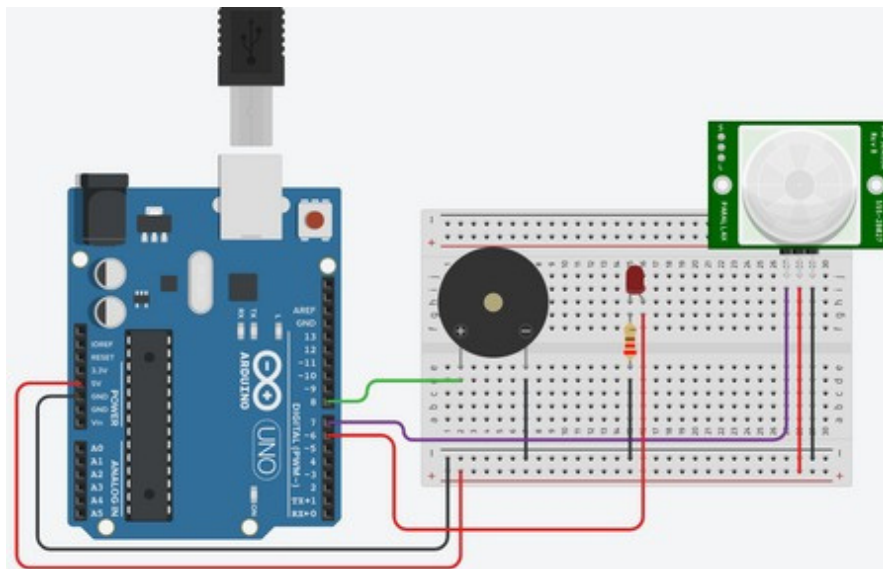


CÓDIGO EN BLOQUES



ALARMA

- ✓ Arduino UNO
- ✓ Sensor PIR(sensordemovimiento)
- ✓ LED rojo (para indicarmovimiento)
- ✓ Buzzer
- ✓ Resistencias:
 - 220 Ω para el LED
- ✓ Jumpers (cables de conexión)
- ✓ Protoboard (opcional, para mayor orden)



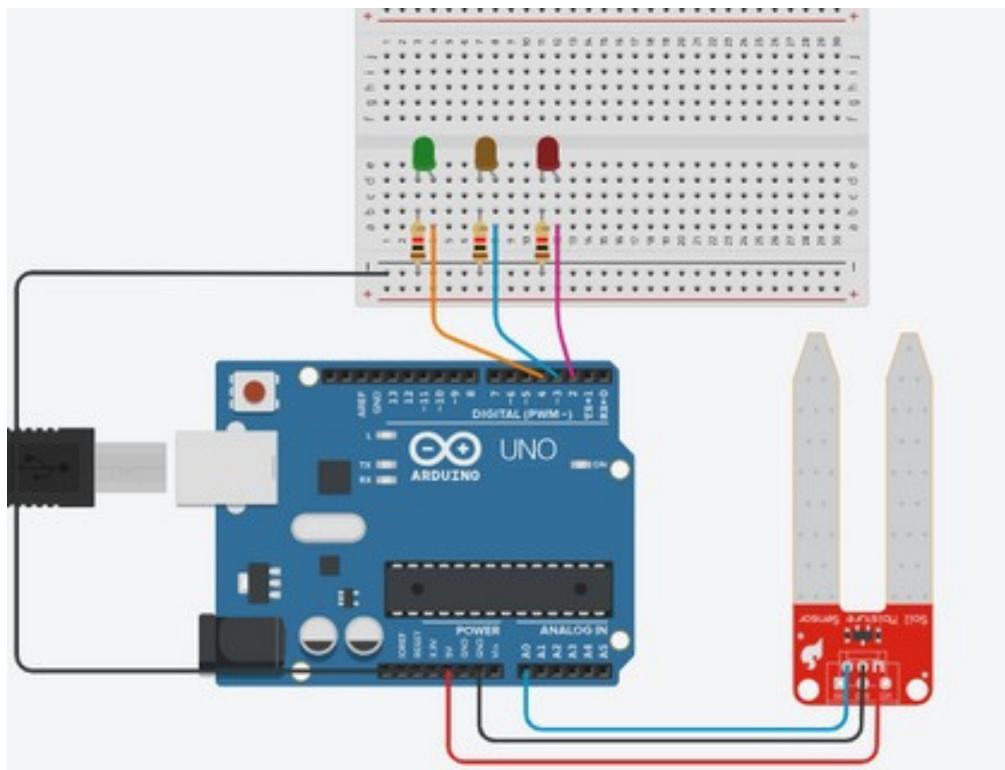
CÓDIGO EN BLOQUES



SENSOR DE HUMEDAD

Materiales necesarios (en Tinkercad):

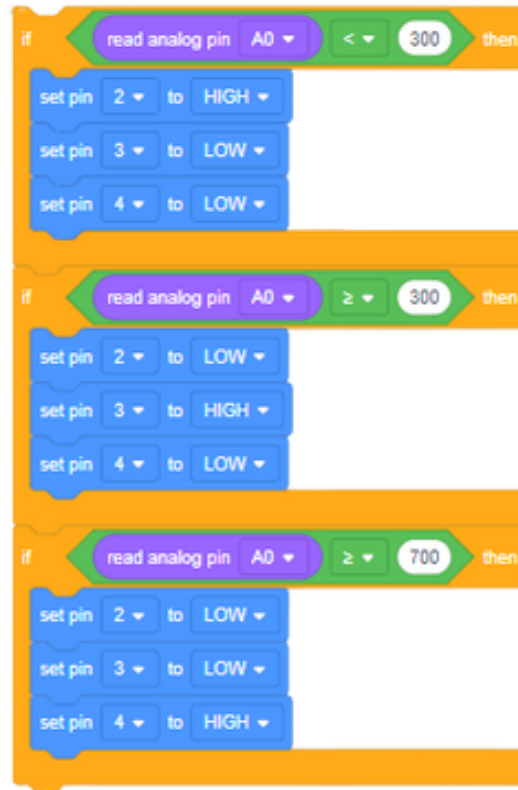
- ✓ Arduino UNO
- ✓ Sensor de humedad del suelo (soil moisture sensor)
- ✓ LED verde (para nivel de humedad alto)
- ✓ LED amarillo (para nivel de humedad medio)
- ✓ LED rojo (para nivel de humedad bajo)
- ✓ Resistencias (220 Ω para cada LED)
- ✓ Jumpers (cables de conexión)
- ✓ Protoboard (opcional, para mayor orden)



CÓDIGO EN BLOQUES



TECNOMAKETERS



CÓDIGO EN TEXTO

```
// C++ code
//
void setup()
{
  pinMode(A0, INPUT);
  pinMode(2, OUTPUT);
  pinMode(3, OUTPUT);
  pinMode(4, OUTPUT);
}
void loop()
{
  if (analogRead(A0) < 300) {
    digitalWrite(2, HIGH);
    digitalWrite(3, LOW);
    digitalWrite(4, LOW);
  }
  if (analogRead(A0) >= 300) {
    digitalWrite(2, LOW);
    digitalWrite(3, HIGH);
    digitalWrite(4, LOW);
  }
  if (analogRead(A0) >= 700) {
    digitalWrite(2, LOW);
    digitalWrite(3, LOW);
    digitalWrite(4, HIGH);
  }
  delay(10); // Delay a little bit to improve simulation performance
}
```

VOCABULARIO

TECNICO Y ESPECÍFICO

Conexiones y componentes eléctricos

- Protoboard: Placa para realizar conexiones eléctricas sin necesidad de soldar. Ideal para pruebas y prototipos.
- Pin: Punto de conexión física de un componente electrónico, sensor o microcontrolador.
- GND (Ground): Terminal de tierra o referencia cero de voltaje en un circuito.
- VCC / 5V / 3.3V: Terminales de alimentación positiva de un circuito. 5V y 3.3V indican el voltaje entregado por el Arduino.
- Resistencia: Componente que limita el paso de corriente eléctrica para proteger otros componentes.
- Cátodo y Ánodo: Terminales negativo (cátodo) y positivo (ánodo) de componentes como LEDs.
- Polaridad: Dirección en que debe conectarse un componente para que funcione correctamente.

Actuadores y sensores

- Sensor PIR (Passive Infrared): Sensor que detecta movimiento a través de la variación de calor (infrarrojo) en el entorno.
- Actuador: Dispositivo que realiza una acción (como encender un LED o mover un motor) como respuesta a una señal.
- Buzzer: Dispositivo que emite un sonido al recibir corriente. Puede ser pasivo (requiere frecuencia) o activo (solo necesita voltaje).
- LED (Diodo Emisor de Luz): Componente que emite luz cuando circula corriente en la dirección correcta.

VOCABULARIO

Programación y lógica

- Bloques: Elementos visuales utilizados para programar de forma gráfica (tipo Scratch o Tinkercad Blocks).
- C++: Lenguaje textual de programación utilizado en Arduino.
- Condicional (if/else): Estructura lógica que ejecuta diferentes acciones según se cumpla o no una condición.
- Entrada (input): Señal o dato recibido por un pin del Arduino (por ejemplo, desde un sensor).
- Salida (output): Señal enviada desde un pin del Arduino hacia un actuador (como un LED o motor).
- Delay: Retardo programado que pausa la ejecución por cierta cantidad de milisegundos.

Pensamiento computacional y robótica

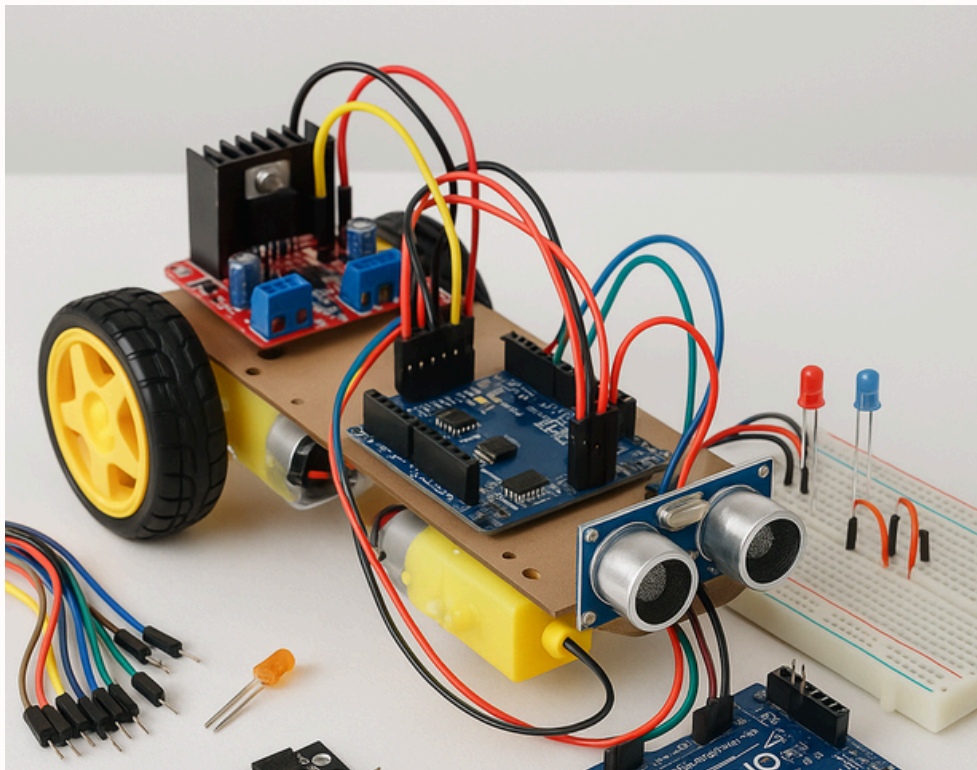
- Algoritmo: Conjunto de pasos ordenados para resolver un problema o realizar una tarea.
- Prototipo: Primera versión funcional de un proyecto, usada para pruebas y mejoras.
- Simulación: Representación virtual de un circuito que permite probar su funcionamiento sin armarlo físicamente.
- Microcontrolador: "Cerebro" del circuito; ejecuta instrucciones y controla entradas y salidas (en este caso, el Arduino).

Entorno de trabajo y herramientas

- Tinkercad: Plataforma online que permite simular circuitos electrónicos y programarlos con bloques o texto.
- Arduino UNO: Placa de desarrollo basada en un microcontrolador, ampliamente usada en educación para aprender programación y electrónica.
- IDE de Arduino: Entorno de desarrollo que permite escribir, cargar y probar código C++ en placas Arduino.

Proyecto Robot Evita Obstáculos

¿Te imaginás que puedas construir un robot que se mueva solo y esquive obstáculos, tal como los autos autónomos?

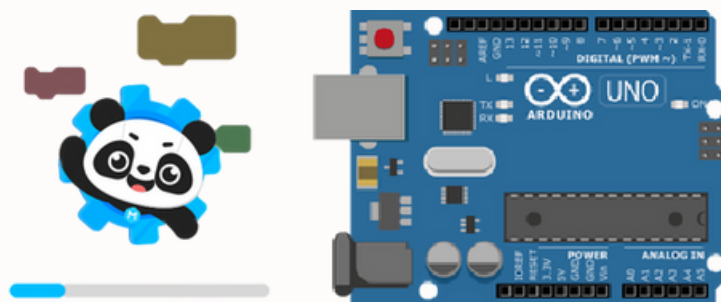


Programamos con mBlock

docentestecnomaketer

Este proyecto no es solo un robot: es una oportunidad para que los chicos integren electrónica, programación y pensamiento computacional en una aplicación concreta y divertida.

Aclaración importante: El Proyecto Robot Evita Obstáculos requiere armado físico con Arduino, motores, sensor ultrasónico y controlador de motor. ⚡ No es posible simular todo el circuito en Tinkercad, ya que esta plataforma no incluye drivers de motor ni permite simular completamente el movimiento. ✅ Lo que sí se puede hacer es diseñar y probar la lógica del código en mBlock, y luego transferirlo a la placa Arduino para que el robot funcione en la realidad como se explica en el documento. PARA ESO USAREMOS MBLOCK Y ARDUINO UNO



Objetivos y Contenidos

DESARROLLO DEL PROYECTO

Objetivos de aprendizaje

- Comprender cómo funcionan los **sensores ultrasónicos / infrarrojos** en un robot.
- Desarrollar un circuito con **placa Arduino, motores y protoboard**.
- Programar el comportamiento de un robot que detecta y evita obstáculos.
- Aplicar la **lógica condicional** ("si pasa esto, entonces hago aquello").
- Realizar la transición de la **programación en bloques (nBlock)** al **lenguaje textual (Arduino C++)**.

Contenidos sugeridos

- Introducción al **sensor ultrasónico**.
- Conexión básica de **motores DC y driver L298N (o similar)**.
- Condicionales y bucles en programación.
- De bloques a código: **comparación visual** (lado a lado).
- Buenas prácticas: comentar el código, modularizar funciones básicas.

Materiales y recursos necesarios

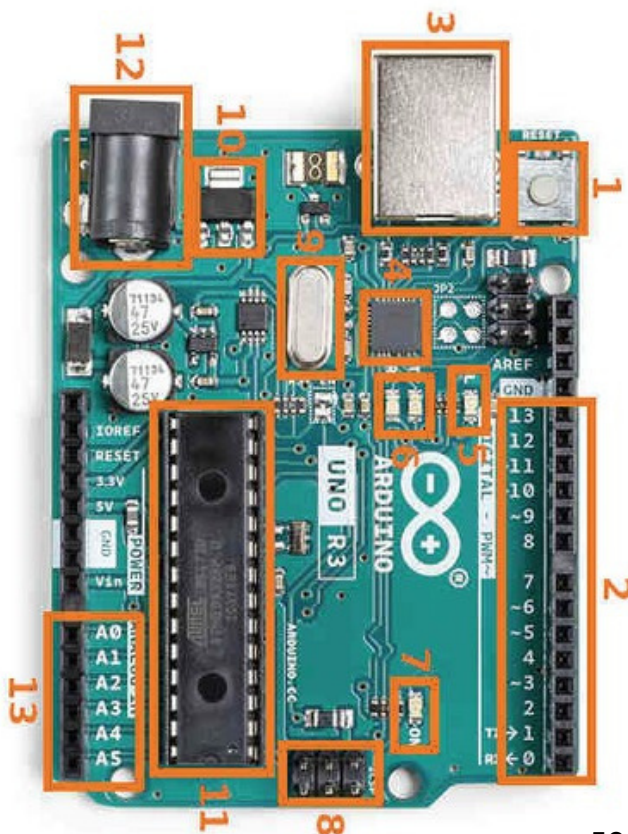
- Arduino UNO (o compatible).
- 2 motores DC con ruedas + soporte.
- Sensor ultrasónico (HC-SR04).
- Driver de motor (ej. L298N).
- Protoboard, cables y batería/pila recargable.
- PC con **mBlock** (bloques) y Arduino IDE (texto).

Actividades paso a paso para armarlo

Cómo funciona el robot: Lógica del Robot Evita Obstáculos

1. Sensor ultrasónico HC-SR04 (pines 2 y 3):
 - a. El pin 3 es el TRIG (manda un pulso de sonido).
 - b. El pin 3 es el ECHO (recibe el rebote del pulso).
 - c. Con esto se mide la distancia hasta el obstáculo
2. Motores (con controlador tipo L298N o similar):
 - a. Motor izquierdo:
 - i. Pin 5 → PWM (velocidad con analogWrite)
 - ii. Pin 6 → Dirección 1
 - iii. Pin 7 → Dirección 2
 - b. Motor derecho:
 - i. Pin 10 → PWM (velocidad con analogWrite)
 - ii. Pin 9 → Dirección 1
 - iii. Pin 8 → Dirección 2
3. Velocidad fija en 200 (sobre 255): El robot avanza mientras no haya obstáculos cerca.
4. Decisión: Si la distancia > 10 cm → el robot avanza. Si la distancia ≤ 10 cm → el robot se detiene y gira para esquivar (por ejemplo, gira a la izquierda).

Los pines...



1. Botón reset
2. Pines digitales E/S
3. Interfaz USB
4. Chip de comunicación serie
5. LED conectado al pin 13
6. LEDs TX y RX
7. LED de encendido
8. ICSP
9. Oscilador de cristal de 16MHz
10. Regulador de tensión
11. Microcontrolador ATmega328P
12. Conector de alimentación
13. Pines analógicos entrada

Conectá tu circuito

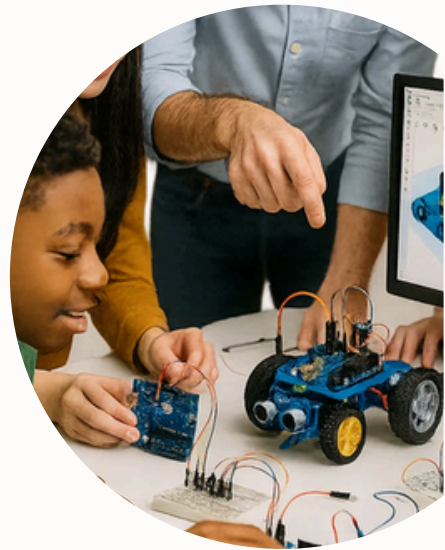
Armado del circuito

En este paso vamos a conectar cada componente para darle vida al robot. Seguí la imagen como guía y verificá siempre que cada cable esté en el pin correcto:

- Sensor ultrasónico HC-SR04
 - TRIG → pin 3
 - ECHO → pin 2
 - VCC → 5V
 - GND → GND
- Controlador de motores (L298N o similar)
 - Motor derecho:
 - ENA → pin 5 (PWM, controla velocidad)
 - IN4 → pin 6
 - IN3 → pin 7
 - Motor izquierdo:
 - ENB → pin 10 (PWM, controla velocidad)
 - IN2 → pin 8
 - IN1 → pin 9
 - Alimentación de motores: conectar a batería externa (mínimo 7.4V, máximo 12V según tu módulo).
 - GND del módulo debe estar unido al GND del Arduino para que todo comparta la misma referencia eléctrica.

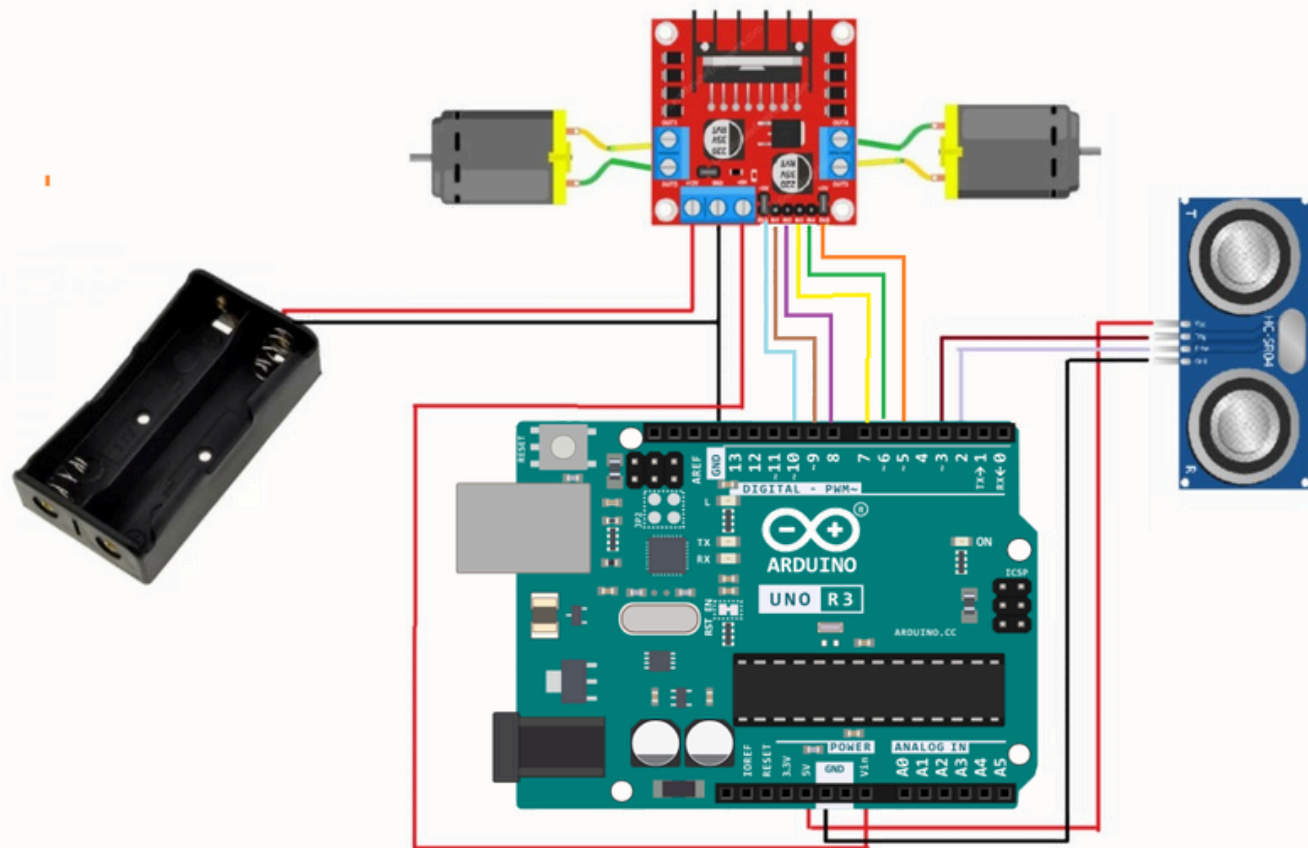
⚡ **Consejo:** ir conectando componente por componente y probar cada parte (por ejemplo, primero el sensor, luego un solo motor) antes de montar todo el circuito. Así reducen errores y entienden cómo cada pieza cumple su función.

✅ Cuando el circuito esté listo, tu robot ya estará preparado para recibir el código y comenzar a moverse.



¡IMPORTANTE! Verifica todas las conexiones antes de encender el robot. Una conexión incorrecta podría dañar los componentes.

Seguí este modelo



Programar en Bloques

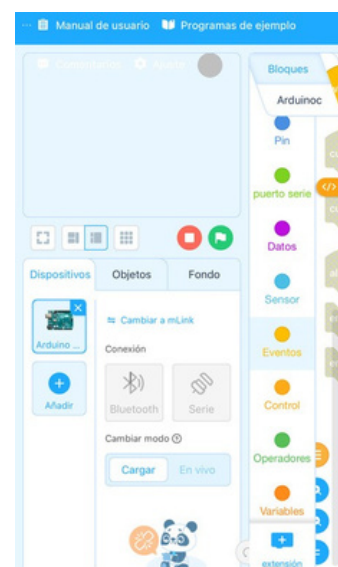


Abrir MBlock y crear un nuevo proyecto

1. Ingresá a MBlock desde la aplicación de escritorio o el navegador.
2. Si no tenés cuenta, registrate con tu correo y contraseña (puede ser Google o correo propio).
3. Iniciá sesión.
4. Una vez dentro, hacé clic en “Crear” para iniciar un nuevo proyecto.

Seleccionar el tipo de dispositivo

1. Hacé clic en “Dispositivo” + (más)
2. Buscá tu placa de Arduino
3. Seleccioná el modelo correcto. Esto hará que MBlock cargue los bloques y funciones compatibles con tu robot.



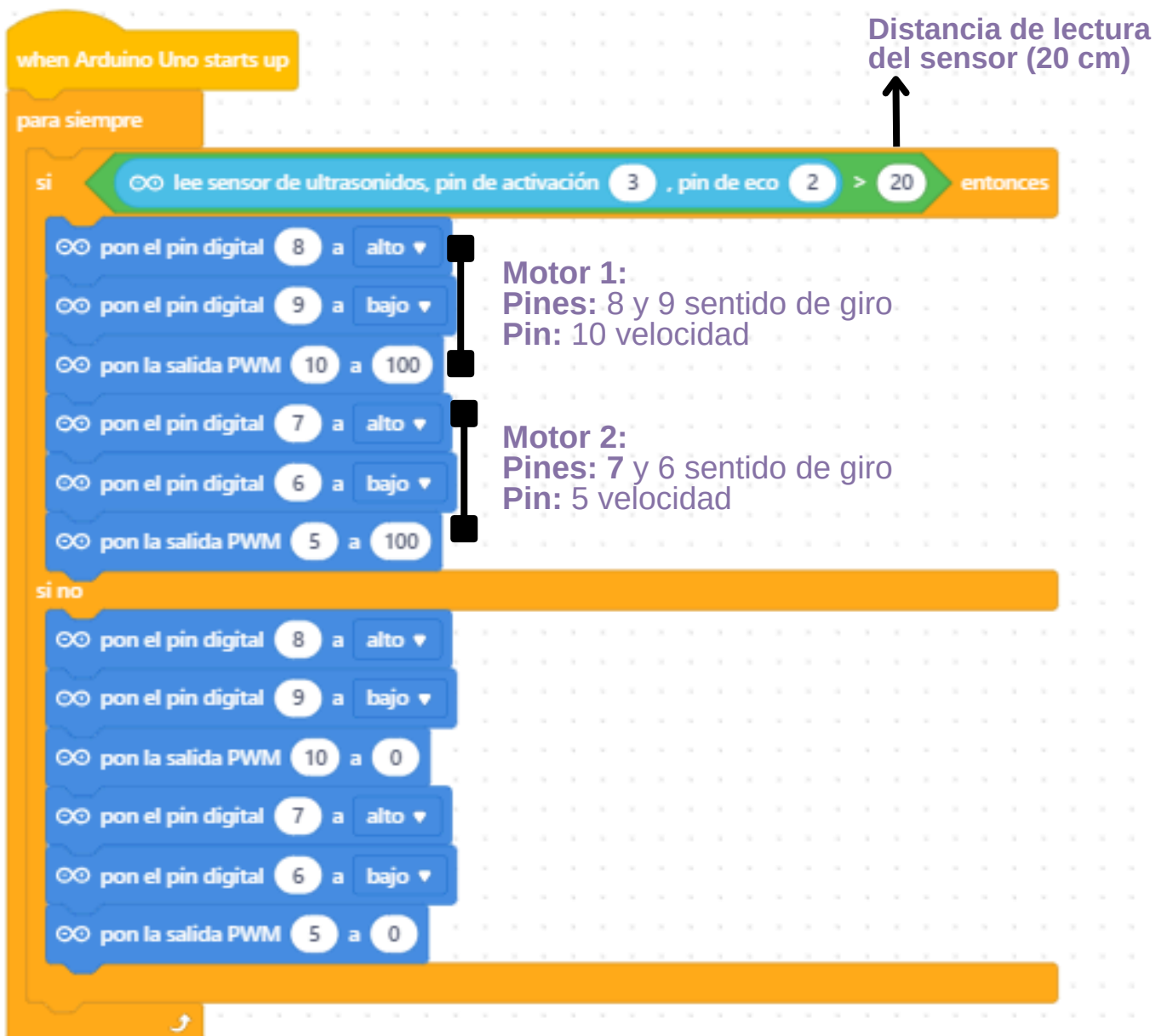
Conectar el robot un el puerto USB

1. Conectá tu Arduino al computador con el cable USB.
2. En MBlock, hacé clic en “Conectar” → “Dispositivo por USB”.
3. Esperá a que indique que el dispositivo está conectado correctamente.

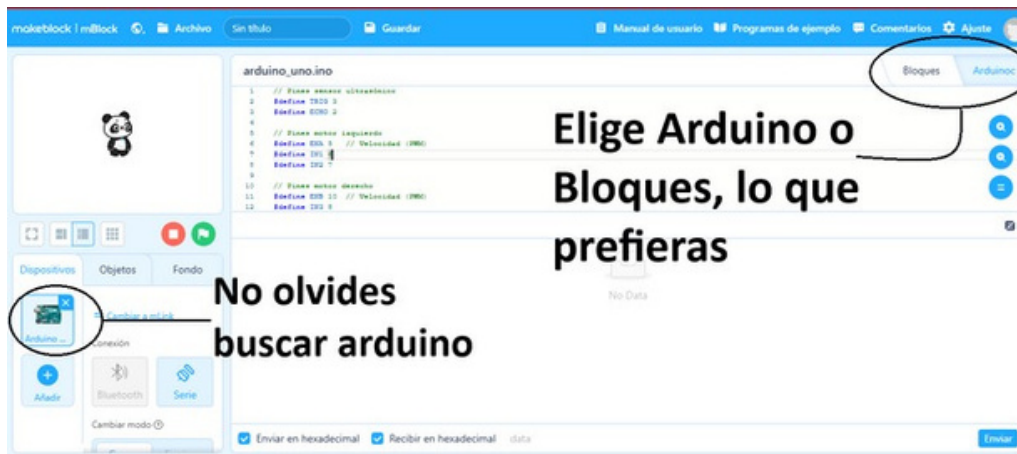
Familiarizarse con los bloques

Programa en Bloques

1. En el código por bloques se utilizaron:
2.  Bloques amarillos (Control y Eventos): permiten iniciar la ejecución y usar condicionales del tipo “si... entonces”.
3.  Bloques azules (Movimiento/Salidas): se encargan de encender los motores y definir su dirección. además, (PWM – control de velocidad): permiten regular la potencia enviada a los motores para avanzar o retroceder de manera controlada.
4.  Bloques verdes (Operadores/Matemática): procesan comparaciones, por ejemplo, “si la distancia es menor a 10 cm”.
5.  Bloques celestes (Sensores): corresponden al ultrasonido, devuelven la distancia medida en centímetros.



Programa en Texto



Se desarrolla en dos partes ya que es muy extenso. Usando la opción de Mblock Arduino se puede escribir el código en texto. Luego conectar nuestra arduino y subirlo...

```
// Pines sensor ultrasónico
#define TRIG 3
#define ECHO 2

// Pines motor izquierdo
#define ENA 5 // Velocidad (PWM)
#define IN1 6
#define IN2 7

// Pines motor derecho
#define ENB 10 // Velocidad (PWM)
#define IN3 8
#define IN4 9

int velocidad = 200; // Velocidad fija (0-255)

void setup() {
  // Configuración sensor ultrasónico
  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);

  // Configuración motores
  pinMode(ENA, OUTPUT);
  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);

  // Configuración motores
  pinMode(ENB, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);

  Serial.begin(9600);
}

void loop() {
  long distancia = medirDistancia();
  Serial.print("Distancia: ");
  Serial.println(distancia);

  if (distancia > 10) {
    avanzar();
  } else {
    retroceder();
    delay(500); // retrocede medio segundo antes de volver a medir
  }
}
```

Link del código INO
LO PODES DESCARGAR
Y PONER EN TU
ARDUINO IDE



https://drive.google.com/file/d/1xIh7ivFFim4NDUewnl10-8lviXtYGpyb/view?usp=drive_link

Parte 2 del código

```
// Función para medir distancia
long medirDistancia() {
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    long duracion = pulseIn(ECHO, HIGH);
    long distancia = duracion * 0.034 / 2; // cm
    return distancia;
}

// Movimiento: Avanzar
void avanzar() {
    analogWrite(ENA, velocidad);
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);

    analogWrite(ENB, velocidad);
    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
}

// Movimiento: Retroceder
void retroceder() {
    analogWrite(ENA, velocidad);
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);

    analogWrite(ENB, velocidad);
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
}
```

Evaluación y reflexión

Documentar en una **bitácora**:

- ¿Qué funcionó bien?
- ¿Qué parte costó más?
- ¿Cómo lo mejorarías?

Actividad de extensión: agregar un LED que se prenda al detectar obstáculos.

Reflexión STEAM: relacionar con autos autónomos, sensores de proximidad en celulares, robótica industrial.

“Lo que empezó como un LED y una resistencia, ahora se convirtió en un robot autónomo. El siguiente paso depende de vos y tus estudiantes.”

✓ Rúbrica de evaluación – Armado de Robot Evita-Obstáculos

Criterio 1 – Conexiones electrónicas (0-3 pts)

- 3 pts: Todas las conexiones están correctas y organizadas (sensores, pines, motores).
- 2 pts: Una conexión menor con error, pero el circuito funciona.
- 1 pt: Varias conexiones mal hechas, el circuito no funciona correctamente.
- 0 pts: No logra conectar el circuito.

Criterio 2 – Programación en bloques (0-3 pts)

- 3 pts: El código usa correctamente condicionales, sensores y PWM; el robot responde como se espera.
- 2 pts: El código funciona con errores menores (ej. velocidad mal calibrada).
- 1 pt: Se reconocen intentos pero no cumple con la lógica esperada.
- 0 pts: No presenta código o no funciona.

Criterio 3 – Trabajo en equipo y organización (0-2 pts)

- 2 pts: Colaboró activamente, resolvió problemas en conjunto.
- 1 pt: Participación mínima o poco organizada.
- 0 pts: No colaboró o interrumpió el trabajo.

Criterio 4 – Presentación y reflexión (0-2 pts)

- 2 pts: Explica con claridad cómo armó el robot y qué ajustes debió hacer.
- 1 pt: Explicación incompleta o poco clara.
- 0 pts: No presenta reflexión.

Total: 0 – 10 puntos

Anexo

Recomendaciones para el Proyecto Robot Evita Obstáculos - (Recomendación docente)

1. Alimentación y baterías (tip clave)

⚡ Para que el robot funcione correctamente, **las baterías deben proveer más de 5 voltios.**

Se recomienda usar **baterías recargables de 3,7 V (tipo 18650)** conectadas en serie, alcanzando un voltaje total de **7,2 V.**

Este voltaje se envía primero al **controlador de motores L298N**, que a su vez alimenta a los motores y también entrega corriente al Arduino para que pueda procesar la lectura de los sensores.

2. Autonomía del robot

🔋 Usar baterías recargables no solo es más económico, sino también más sustentable. Una carga completa suele dar entre **40 y 60 minutos de autonomía**, dependiendo del peso del robot y la fricción de las ruedas.

3. Precauciones de conexión

⚠ Siempre conectar primero el **driver L298N** antes de alimentar directamente al Arduino. Esto evita picos de tensión que puedan dañar la placa.

⚠ **JAMAS INTENTES CONTROLAR DIRECTAMENTE LOS MOTORES AL ARDUINO SIN CONTROLADOR**

4. Alternativa de bajo costo

💡 Si no disponés de baterías recargables, se pueden usar **pilas AA** (4 en serie = 6V), aunque su duración será menor y podrían limitar el rendimiento de los motores.

5. Extra STEAM (Ampliación interdisciplinaria)

📐 Aprovechá la parte de alimentación para vincular con Matemática y Física:

- **Cálculo de voltajes en serie**
- **Relación voltaje-corriente-potencia (Ley de Ohm)**
- **Comparación de eficiencia entre pilas comunes y baterías recargables**



© 2025 Todos los derechos reservados.

Proyecto educativo: Robótica Docente Pro

Este material forma parte de un proyecto educativo independiente destinado a docentes que desean iniciarse en robótica de forma simple, guiada y didáctica.

Licencia de uso

Este ebook está protegido por derechos de autor y es para uso personal e intransferible.

Está estrictamente prohibida la redistribución, reproducción, publicación, venta, copia o cualquier uso comercial de este contenido sin autorización expresa y por escrito

El incumplimiento de estas condiciones puede implicar acciones legales bajo la Ley de Propiedad Intelectual vigente.

